Portfolio

Jakob Sverre Alexandersen
GRA4158 Marketing and the Analysis of Experiments and Quasi-Experiments

April 15, 2026

Abstract

This portfolio presents solutions to four assignments covering the design and analysis of experiments and quasi-experiments in marketing research. Task 1 estimates the causal effect of digital advertising on purchase conversion using a randomized experiment, addressing non-compliance through IV estimation and assessing ad profitability. Task 2 explores a potential study that examines whether companies can relax their marketing budgets by utilizing more "normal" people instead of "high-maintenance" influencers to promote their product, and thus attracting more customers. Task 3 estimates the causal effect of a discount on sales from an observational dataset, employing regression adjustment, propensity score matching, Mahalanobis distance matching, and inverse probability weighting under the conditional independence assumption. Task 4 evaluates the impact of a new in-store marketing strategy on weekly product sales using panel data from 21 stores, applying pre-post analysis, two-way fixed effects, difference-in-differences, synthetic control, and synthetic DID to a single treated store.

Please note that all code is included in the appendix, and will be referenced throughout the paper.

Contents

1 Task 1: The Gen-Z Intern **3**
1.1 Non-Compliance Detective . . . 3
1.2 Treatment effect estimates . . . 4
1.3 Impressive Impressions . . . 5
1.4 Is showing ads profitable? . . . 7
1.5 Discussion of methods: . . . 8

2 Task 2: Just Buy This, I Don't Know What Else to Tell You **10**
2.1 Introduction & Research Question . . . 10
2.2 Variables and Operationalization . . . 10
2.2.1 Independent Variable . . . 10
2.2.2 Counterfactual and the Compound Treatment Problem . . . 11
2.2.3 Dependent Variable . . . 11
2.2.4 Challenges in Manipulation and Measurement . . . 11
2.3 Experimental Design Strategy . . . 12
2.3.1 Between-Subjects Design . . . 12
2.3.2 Lab vs. Field Experiment . . . 12
2.4 Validity Analysis . . . 13
2.4.1 Threats to Internal Validity . . . 13
2.4.2 Threats to External Validity . . . 14
2.5 Discussion and Conclusion . . . 14

3 Task 3: The Return of the Gen-Z Intern **16**
3.1 EDA . . . 16
3.2 Regression Adjustment . . . 16
3.3 Matching . . . 17
3.3.1 Propensity Score Estimation and Common Support . . . 17
3.3.2 Propensity Score Matching . . . 17
3.3.3 Mahalanobis Distance Matching . . . 18
3.4 Inverse Probability Weighting (IPW) . . . 18
3.5 Augmented Inverse Probability Weighting (AIPW) . . . 18
3.6 Double ML . . . 19
3.7 Comparison and Discussion . . . 19
3.7.1 Why do the estimates differ? . . . 19
3.7.2 Which estimates do we trust the most? . . . 19
3.7.3 Assumptions and limitations . . . 20
3.7.4 Recommendation to the marketing manager . . . 20

4 Task 4: The sales analyst **21**
4.1 Exploratory Data Analysis . . . 21
4.2 Pre-Post Analysis (store 109 only) . . . 22
4.3 Two-Way Fixed Effects Panel Model . . . 23
4.4 Difference-in-Differences . . . 24
4.5 Synthetic Control Method . . . 25
4.6 Demeaned SCM . . . 27
4.7 Synthetic DID . . . 29
4.8 Which methods should we trust most? . . . 31

A Code **34**
A.1 Task 1 Code . . . 34
A.2 Task 3 Code . . . 41
A.3 Task 4 Code . . . 50

1 Task 1: The Gen-Z Intern

1.1 Non-Compliance Detective

In order to check for non-compliance, we need to see if the treatment group (test = 1) should see ads (impressions > 0), and the control group should NOT see ads (impressions = 0 in practice)

Since the control group has "ghost ads" (hypothetical impressions), non-compliance would be if the control group actually SAW ads.

```
treatment_group = df[df['test'] == 1]
control_group = df[df['test'] == 0]

treatment_no_exposure = treatment_group[treatment_group['impressions'] == 0]
treatment_with_exposure = treatment_group[treatment_group['impressions'] > 0]

compliance_rate_treatment = len(treatment_with_exposure) / len(treatment_group)
non_compliance_rate = len(treatment_no_exposure) / len(treatment_group)
```

```
> treatment group size: 25072
> treatment group with impressions > 0 (compliers): 19694
> treatment group with impressions = 0 (non-compliers): 5378
> compliance rate in treatment: 0.7855 (78.55%)
> non-compliance rate: 0.2145 (21.45%)
> control group with 0 ghost impressions: 0.2159017971758665
```

The data shows one-sided non-compliance. In the treatment group, 21.45% were assigned to receive ads but had 0 impressions (the never-takers). The control group shows perfect compliance. The ghost ads system ensures that the control group never actually saw ads. The 21.59% with 0 ghost impressions simply indicates users who would not have been exposed regardless of assignment. This is <u>not</u> two-sided non-compliance because no one in the control group actually received treatment. The full code and outputs for this task is provided in Appendix A.1

We are able to estimate ITT because we have a simple comparison of means by assignment (test = 0 vs test = 1). We cannot observe counterfactual for never-takers who may systematically differ, so we cannot estimate ATE. A naive ATET estimate (comparing those with impressions > 0 in treatment vs. controls) would be biased because never-takers may systematically differ from compliers. Without additional assumptions, ATET is not cleanly identified. LATE is also estimable since using assignment as an instrument for actual exposure estimates effect for compliers. Since we have one-sided non-compliance with perfect compliance in control, LATE = ATET for compliers, which is actually quite informative for managerial decisions.

1.2 Treatment effect estimates

The difference in conversion rates between assignment groups is 0.465 percentage points ($p = 0.001$, 95% CI : [0.2pp, 0.7pp]). This represents the effect of being *assigned* to see ads, regardless of actual exposure. The control group conversion rate was 2.4% and the treatment group was 2.87%.

```
#bin var treatment indicator (actually saw ads)
df['saw_ads'] = ((df['test'] == 1) & (df['impressions'] > 0)).astype(int)

#ITT
itt_control = df[df['test'] == 0]['purchase'].mean()
itt_treatment = df[df['test'] == 1]['purchase'].mean()
itt_effect = itt_treatment - itt_control

print(f'control group conversion rate: {itt_control}')
print(f'treatment group conversion rate: {itt_treatment}\nITT effect: {itt_effect}
    ')

#ITT with regression (for st err)
X_itt = sm.add_constant(df['test'])
model_itt = sm.OLS(df['purchase'], X_itt).fit(cov_type='HC1') #robust SEs
print(f'regression:\n{model_itt.summary().tables[1]}')
```

```
> control group conversion rate: 0.024029204107830552
> treatment group conversion rate: 0.028677409061901724
> ITT effect: 0.004648204954071172
> regression:
> ==============================================================================
>                 coef    std err          z      P>|z|      [0.025      0.975]
> ------------------------------------------------------------------------------
> const          0.0240      0.001     24.773      0.000       0.022       0.026
> test           0.0046      0.001      3.245      0.001       0.002       0.007
> ==============================================================================
```

LATE: Using random assignment as an instrument for actual ad exposure, we estimate the effect of *actually seeing* ads for compliers is 0.59 percentage points ($p = 0.001$, CI: [0.23pp, 0.95pp])

```
#LATE via IV
#instrument: test, endo: saw_ads, outcome: purchase

df_iv = df[['purchase', 'saw_ads', 'test']].copy()
df_iv['const'] = 1

iv_model = IV2SLS(
    dependent=df_iv['purchase'],
    exog=df_iv['const'],
    endog=df_iv['saw_ads'],
    instruments=df_iv['test'],
).fit(cov_type='robust')

iv_model.summary
```

```
> IV-2SLS Estimation Summary
> Dep. Variable: purchase R-squared: 0.0015
> Estimator: IV-2SLS Adj. R-squared: 0.0015
> No. Observations: 50000 F-statistic: 10.544
> Date: Wed, Feb 04 2026 P-value (F-stat) 0.0012
> Time: 17:58:29 Distribution: chi2(1)
> Cov. Estimator: robust
> Parameter Estimates
> Parameter Std. Err. T-stat P-value Lower CI Upper CI
> const 0.0240 0.0010 24.774 0.0000 0.0221 0.0259
> saw_ads 0.0059 0.0018 3.2471 0.0012 0.0023 0.0095
```

Interpretation: For every 1,000 users who see the ad, we expect approximately 6 additional conversions compared to not showing ads. The LATE is larger than the ITT because the ITT is diluted by never-takers (users assigned to treatment who never visited the site).

Note: We do not report an ATET estimate because it would suffer from selection bias, as mentioned in task 1.

1.3 Impressive Impressions

```python
#create impression bins for strat analysis
df['imp_bins'] = pd.cut(df['impressions'], bins=[-1, 0, 1, 2, 3, 5, 10, 20],
    labels=['0', '1', '2', '3', '4-5', '6-10', '11+'])

#calc ITT bu imp bin
cate_by_bin = df.groupby(['imp_bins', 'test'])['purchase'].agg(['mean', 'count', '
    std']).unstack()
cate_by_bin.columns = ['_'.join(map(str, col)) for col in cate_by_bin.columns] #
    convert to str before joining
cate_by_bin['ITT'] = cate_by_bin['mean_1'] - cate_by_bin['mean_0']
cate_by_bin['n_total'] = cate_by_bin['count_0'] + cate_by_bin['count_1']

cate_by_bin[['mean_0', 'mean_1', 'ITT', 'n_total']].round(4)
```

```
>       mean_0 mean_1 ITT   n_total
> imp_bins
> 0     0.0000 0.0000 0.0000 10760
> 1     0.0000 0.0000 0.0000 2932
> 2     0.0000 0.0000 0.0000 5860
> 3     0.0000 0.0000 0.0000 7830
> 4-5 0.0000 0.0001 0.0001 14039
> 6-10 0.1302 0.1564 0.0262 8478
> 11+ 1.0000 1.0000 0.0000 101
```

```python
#regression with interaction to test heterogeneity
#model: purchase ~ test + impressions + test*impressions

df['test_x_imp'] = df['test'] * df['impressions']
X_interact = sm.add_constant(df[['test', 'impressions', 'test_x_imp']])
model_interact = sm.OLS(df['purchase'], X_interact).fit(cov_type='HC1')

print(model_interact.summary().tables[1])
```

```
> ==============================================================================
>     coef    std err          z      P>|z|      [0.025      0.975]
> ------------------------------------------------------------------------------
> const          -0.0533      0.002    -27.571      0.000      -0.057      -0.050
> test           -0.0067      0.003     -2.420      0.016      -0.012      -0.001
> impressions     0.0242      0.001     28.343      0.000       0.023       0.026
> test_x_imp      0.0035      0.001      2.866      0.004       0.001       0.006
> ==============================================================================
> - base effect of treatment (at 0 impressions): -0.0066651281558449925
> - additional effect per impression: 0.003506070910565647
> - interaction significant? p = 0.004154874372134771
```

The interaction model reveals significant heterogeneity in treatment effects ($p = 0.004$). The effect of ads increases with the number of impressions:

- 0-5 impressions: near-zero treatment effect (ITT ≈ 0)
- 6-10 impressions: ITT = 2.62 percentage points, substantially higher than the overall average
- Per-impression effect: each additional impression adds approximately 0.35 percentage points to the treatment effect

This suggests a "threshold effect", ads need repeated exposure before influencing purchase behavior. The practical implication is that low-frequency exposure may not be cost-effective; the company should prioritize reaching users with 6+ impressions to maximize ROI.

These findings supplement our earlier estimates by showing that the overall ITT (0.46pp) and LATE (0.59pp) are *averages across heterogeneous effects*. The true effect for high-exposure users is considerably larger.

1.4 Is showing ads profitable?

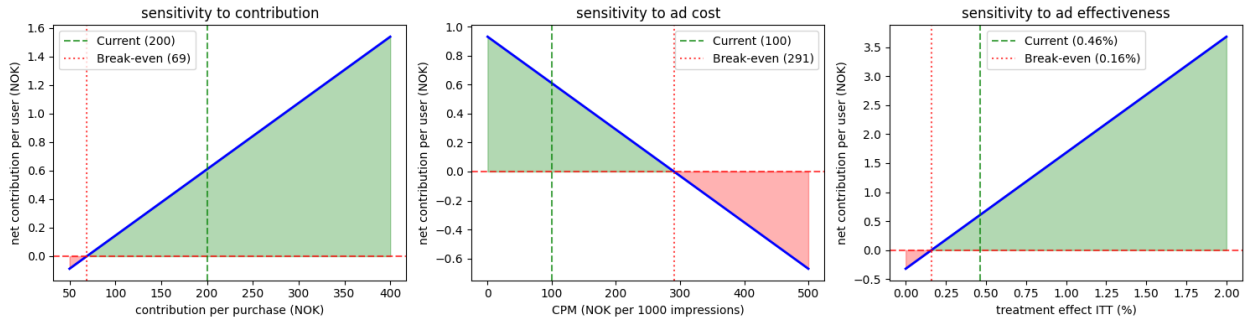
```python
contribution_per_purchase = 200
cost_per_impression = 100 / 1_000

late_effect = 0.0059
itt_effect = 0.00465

avg_impressions_treatment = df[df['test'] == 1]['impressions'].mean()
avg_impressions_control = df[df['test'] == 0]['impressions'].mean()  # ghost impressions
avg_impressions_compliers = df[(df['test'] == 1) & (df['impressions'] > 0)]['impressions'].mean()
revenue_lift_per_user = itt_effect * contribution_per_purchase
cost_per_user = avg_impressions_treatment * cost_per_impression
net_contribution_per_user_itt = revenue_lift_per_user - cost_per_user
n_treatment = len(df[df['test'] == 1])
total_impressions_treatment = df[df['test'] == 1]['impressions'].sum()

total_revenue_lift = n_treatment * itt_effect * contribution_per_purchase
total_ad_cost = total_impressions_treatment * cost_per_impression
total_net_contribution = total_revenue_lift - total_ad_cost
```

```
> contribution per purchase: 200 NOK
> cost per impression: 0.1 NOK
> average impressions (treatment): 3.20
> average impressions (compliers only): 4.07
> revenue lift per user: 0.9299999999999999
> ad cost per user: 0.32007418634333124
> net contribution per user: 0.6099258136566688
> ROI: 190.5576393475308
> users in treatment: 25072
> total impressions served: 80249
> expected additional conversions: 116.58479999999999
> total revenue lift: 23316.96 NOK
> total ad cost: 8024.900000000001 NOK
> net contribution: 15292.059999999998 NOK
```

Yes, the advertising campaign is profitable. Using the ITT estimate, each user assigned to treatment generates a revenue lift of 0.93 NOK (0.465% $\times$ 200NOK) at a cost of 0.32 NOK (3.2 impressions $\times$0.10 NOK), yielding a **net contribution of 0.61 NOK per user** and an **ROI of approximately 190%**

At the campaign level, the treatment group of $\sim$25k users generated approximately 117 additional conversions, with a total revenue lift of 23,317 NOK against ad costs of 8,025 NOK, for a **net contribution of 15,292 NOK.**

Sensitivity analysis shows substantial margins of safety; the campaign remains profitable as long as contribution per purchase exceeds 69 NOK (current: 200 NOK). The campaign can tolerate CPM up to 291 NOK before it becomes unprofitable (current: 100 NOK). The treatment effect could fall to 0.16% before breaking even (current: 0.46%). All parameters have approximately 66% margin of safety, indicating that profitability conclusion is robust to measurement uncertainty.



1.5 Discussion of methods:

1. ITT: We estimated the ITT effect using OLS regression of purchase outcomes on treatment assignment, using heteroskedasticity-consistent standard errors. This approach relies on random assignment, which is satisfied by the experimental design. The primary limitation of the ITT estimate is that it represents a "diluted" effect because it includes never-takers who were assigned to treatment but never actually saw ads. Additionally, the linear probability model used for the binary outcome can *in theory* produce predicted probabilities outside the $[0, 1]$ range (although this is rarely problematic for estimating average effects). Alternative approaches include logistic regression (as mentioned in lecture 5), which provide odds-ratios rather than percentage point changes.

2. LATE: To account for non-compliance, we estimated the LATE using two-stage least squares with random assignment as an instrument for actual ad exposure. This approach requires four key assumptions: relevance (assignment must affect actual exposure), the exclusion restriction (assignment affects purchase only through ad exposure), monotonicity (no defiers who would only see ads when assigned to control), and independence (random assignment). Our first stage is strong with a compliance difference of 78.5%, satisfying the relevance condition.

   The main limitation to LATE is that it identifies the causal effect only for compliers (those who see ads when assigned to treatment and don't see ads when assigned to control). This effect may not generalize to never-takers, whose response to ads we cannot observe. The exclusion restriction is fundamentally untestable and could be violated if merely being part of an experiment affects behavior independently of ad exposure. Alternative approaches include bounds analysis, which relaxes assumptions at the cost of wider intervals, or principal stratification methods.

3. CATE: We examined effect heterogeneity by impression level using an interaction model that included treatment, impressions and their product. This approach test whether the treatment effect varies systematically with exposure intensity. The key assumption is that this relationship is linear, which may miss more complex non-linear patterns such as threshold effects or diminishing returns. The ghost ads design is particularly valuable here, as it allows us to compare treatment and control groups at the same hypothetical exposure levels.

   Limitations include the parametric nature of the interaction specification and concerns about multiple comparisons when testing many sub-groups. Small sample sizes at extreme impression levels (the 11+ bin in the code contains only 101 observations) also reduce precision and reliability for those estimates. More flexible alternatives include causal forests or other ML-based methods.

   Our inference relies on asymptotic approximations with robust standard errors to account for heteroskedasticity, which is particularly important for binary outcomes where variance inherently depends on the mean. With 50k observations, large-sample approximations are generally reliable. However, our standard errors do not account for potential clustering — if users share devices or households, observations may not be independent, and cluster-robust standard errors would be more appropriate.

4. SUTVA has two components: no interference between units and no hidden versions of treatment. The first component requires that one user's treatment status does not affect another user's outcome.

This could be violated through word-of-mouth (a user who sees the ad might tell a user in the control group about the ad, influencing their purchase decision), shared devices / households, or social network effects. These inference effects would typically cause us to *underestimate* the true ad effect, because some of the treatment "spills over" to control. The second component assumes that there is only one version of treatment. In this experiment, users could receive different numbers of impressions, which are effectively different "doses" of treatment. Our CATE analysis showed that the treatment effect varies substantially with impression count; users with 6+ impressions showed much larger effects than those with 1-5 impressions. The heterogeneity in treatment intensity means our average estimates (ITT, LATE) mask important variation. The single "treatment effect" is actually a weighted average across very different exposure levels.

5. The exclusion restriction (for IV/LATE) requires that random assignment affects purchases *only through* actual ad exposure. This could be violated through the Hawthorne effect; if users somehow became aware that they were in an experiment and let it affect their behavior. It could also be affected through the tracking cookie placed for randomization; by itself, it might affect user experience or trigger other platform behaviors that influence purchase. In this experiment, the ghost ads design helps satisfy the exclusion restriction by ensuring the control group has comparable tracking to the treatment group. This is a strength of the experimental design. If the exclusion restriction were to be violated, it would bias the LATE estimate. The direction depends on how the violation affects behavior.

6. Monotonicity requires that no user would see ads only when assigned to control. In this design, monotonicity is satisfied by construction; the control group cannot see ads regardless of assignment, and the ghost ads system ensures one-sided non-compliance.

7. Random assignment appears satisfied by the cookie-based randomization. We could verify this by checking for balance on observable pre-treatment characteristics if such data were available. Without additional covariates in the dataset, we rely on the experimental design description.

The most concerning potential violation is interference through word-of-mouth or shared households, which cause us to underestimate the true ad effect. However, in a large-scale digital advertising context with 50,000 users, spillover effects are likely diluted across many social networks, suggesting the bias is probably modest. The heterogeneity in treatment intensity (varying impression) is documented in the CATE analysis rather than being a hidden bias

2 Task 2: Just Buy This, I Don't Know What Else to Tell You

2.1 Introduction & Research Question

The relationship between younger consumers and advertising is undergoing a fundamental shift. Adoption of ad-blocking software among young internet users has grown at approximately 40% year-over-year in the US in 2015 (Sterling, 2015), and eye-tracking research on digital natives reveals that intrusive advertising formats (pop-ups, non-skippable video ads, and content-reorganizing banners) are perceived as deeply annoying and are actively avoided (Balaskas et al., 2024). This is not merely a technological trend; it reflects a generational change in how advertising credibility is evaluated. Djafarova and Rushworth (2017) found that young consumers perceive non-traditional content creators, like bloggers, and YouTubers, as significantly more credible and persuasive than traditional celebrity endorsers or conventional brand advertisements. More recent work confirms that influencer authenticity has a strong positive effect on Gen Z trust and purchase intentions (Gohil et al., 2025), and that Gen Z consumers view authentic influencers as "highly educated friends" from whom they seek advice, rather than as paid advertisers (Singer et al., 2023).

Given this shift, brands are investing heavily in influencer marketing. However, the question of *what style* of influencer content is most effective remains open. On one end of the spectrum, brands commission polished, high-production influencer partnerships with scripted copy and studio-quality visuals. On the other end, a growing number of creators adopt a raw, casual aesthetic: filming on their phone, speaking without a script, and using humor that borders on self-deprecating (e.g., "honestly just buy this, I don't know what else to tell you"). Observational data might suggest that authentic-looking content generates higher engagement, but such observations are heavily confounded: platforms' targeting algorithms amplify content to audiences predicted to engage with it, and users self-select into following creators whose style they prefer. As Braun and Schwartz (2025) demonstrate, even randomized A/B tests conducted through platform-provided tools suffer from *divergent delivery*, where the algorithm serves different ad variants to systematically different user populations, making it impossible to attribute differences to content alone.

This report proposes an experiment to test the following causal question: **does casual, authentic-style influencer content cause higher purchase intent than professionally produced influencer content?** The focal treatment effect concerns the *perceived authenticity* of the content style, not the presence or absence of influencer marketing itself. The remainder of this report specifies the independent and dependent variables, discusses the operationalization of the treatment and counterfactual, evaluates alternative experimental designs, and analyzes threats to internal and external validity.

2.2 Variables and Operationalization

2.2.1 Independent Variable

The theoretical independent variable is content authenticity style; the degree to which influencer content appears spontaneous, unpolished, and genuine versus professionally produced and scripted. This construct is operationalized through two experimental conditions:

The treatment condition presents casual, authentic influencer content. Concretely, this is a short video (approximately 30–60 seconds) filmed on a smartphone in a natural setting (e.g., the influencer's home), with informal language, minimal editing, and a conversational tone. The influencer speaks directly to the camera without a visible script, and the product endorsement is embedded in casual commentary.

The control condition presents professional, polished influencer content. This is a video of identical length featuring *the same influencer* promoting *the same product*, but filmed with studio lighting, high-quality audio, scripted copy, and polished editing resembling a traditional sponsored post.

2.2.2 Counterfactual and the Compound Treatment Problem

The choice to use the same influencer and the same product across both conditions is a deliberate design decision to avoid the *compound treatment problem*. As discussed in the course, a compound treatment arises when the experimental manipulation unintentionally varies multiple factors simultaneously. If we compared influencer A (casual) with influencer B (professional), any observed difference could be attributed to the person, their audience, their delivery style, or the content format, and not solely to authenticity. By holding the influencer and product constant, the manipulation isolates the content style dimension as cleanly as possible.

The counterfactual is an *explicit alternative* rather than the absence of content. This is appropriate because the research question concerns the *relative* effectiveness of two content strategies, not whether influencer content works at all. A "no content" control group would answer a different question and would not provide actionable guidance for the business decision at hand.

2.2.3 Dependent Variable

The primary dependent variable is purchase intent, which captures the consumer's likelihood of purchasing the promoted product after viewing the content. In the proposed field setting, this is operationalized through behavioral measures recorded via a unique tracked link embedded in each post. Each experimental condition (casual and professional) contains a distinct UTM-tagged or affiliate URL pointing to the product page, allowing the researcher to attribute downstream behavior (click-through, add-to-cart, and conversion) to the specific content variant that generated it. These revealed-preference measures avoid the well-documented gap between stated intentions and actual purchasing behavior, providing a direct read on how content style influences real consumer decisions. Were the experiment instead conducted in a lab, the dependent variable would be measured using a validated 7-point Likert scale (e.g., "How likely are you to purchase this product?") supplemented by a hypothetical willingness-to-pay measure. Although they are useful proxies, they are inherently less actionable than observed behavior.

2.2.4 Challenges in Manipulation and Measurement

Two key challenges deserve attention. First, regarding manipulation effectiveness: casual content may vary not only perceived authenticity but also perceived humor, effort, or production quality. In a lab setting, this would be verified through explicit manipulation checks where participants rate perceived authenticity, humor, and production quality. In the proposed field setting, however, such self-report measures are unavailable. The researcher cannot directly confirm that authenticity is the primary dimension being varied, which means the field experiment tests the *bundle* of features that distinguish casual from professional content rather than authenticity in isolation. This is a genuine limitation of the field design and one of the strongest arguments for a complementary lab study to probe the underlying mechanism.

Second, regarding measurement of the DV: although the tracked link provides a clean attribution path from content to behavior, the resulting measures are noisy. A single click does not imply genuine purchase intent, and conversion rates in influencer campaigns are typically low, requiring large sample sizes to detect modest effects. The link also introduces a minor demand artifact: users who notice the tracked URL may behave differently than they would with an organic link, though this concern is mitigated by the fact that UTM parameters and affiliate links are standard practice in influencer content and are unlikely to be noticeable to most users. Furthermore, behavioral data alone cannot reveal *why* participants acted as they did. The mechanism remains a black box. A lab experiment lets researchers measure not just outcomes but also things that might explain them, like perceived authenticity or trust, but this extra detail comes at the cost of realism. A field study, on the other hand, sacrifices some of that detail but relies on the underlying theory to help make sense of the results.

2.3 Experimental Design Strategy

2.3.1 Between-Subjects Design

The proposed experiment uses a between-subjects design, in which each participant is randomly assigned to view either the casual or the professional video, but not both. This choice is driven by the nature of the treatment. A within-subjects design, where the same participant views both videos, would be inappropriate for two reasons. First, it would introduce severe *carryover effects*: once a participant has seen the casual version, their reaction to the professional version (or vice versa) would be contaminated by the comparison. Second, it would create strong *demand effects*, as participants would almost certainly infer that the study is comparing the two styles and adjust their responses accordingly. Both threats are discussed extensively in this course as primary concerns for within-subjects designs.

Between-subjects designs eliminate these concerns at the cost of requiring more participants (since each participant provides data for only one condition) and introducing the possibility of pre-existing differences between groups. However, the latter concern is addressed through random assignment, which ensures that (in theory) the two groups are equivalent on all observed and unobserved characteristics. To further strengthen balance on key demographics (age, gender, social media usage frequency), *stratified random assignment* can be employed, randomizing within homogeneous subgroups rather than across the full sample. This is particularly important if we wish to analyze heterogeneous treatment effects across demographic subgroups.

2.3.2 Lab vs. Field Experiment

The Case for a Lab Experiment

An online lab experiment would offer maximum control over the experimental setting. Each participant could be randomly assigned to view exactly one video within a survey interface, eliminating any interference from platform algorithms. The researcher could include manipulation checks (e.g., "How authentic did this content feel?") and mediator scales (perceived trust, perceived effort) alongside the outcome measures. This diagnostic depth is the core advantage of a lab setting: it allows the researcher not only to estimate the treatment effect but also to probe the mechanism through which it operates. The Hawthorne effect is reduced in an online lab relative to a physical lab, since participants complete the task from their own environment.

However, the lab setting carries substantial limitations for this particular research question. Watching an influencer video inside a survey is fundamentally different from encountering it in a social media feed. The social context (likes, comments, share counts, algorithmic placement among other content) is entirely absent, and these cues may moderate the authenticity effect in practice. Casual content may derive part of its appeal from the *contrast* with polished content in the same feed, an effect that disappears in isolation. Furthermore, the dependent variable in a lab must rely on stated purchase intent rather than actual behavior, introducing the well-documented intention–behavior gap. For a research question whose ultimate business relevance lies in real purchasing outcomes, this is a significant sacrifice.

The Case for a Field Experiment

A field experiment, for instance, running an A/B test through Instagram or TikTok's advertising tools, would score highly on the four key features of field experiments identified in this course: real-world context, authenticity of treatment, representativeness of participants, and relevant outcome measures. Participants would encounter the content organically in their social media feed, respond with real behavior (clicks, purchases), and the sample would reflect the actual target population. The ecological validity of such a design would be high, and the dependent variable could be operationalized as actual click-through, add-to-cart, or conversion. These are behavioral measures that directly answer the business question.

The primary internal validity threat of a field experiment is *divergent delivery*. As Braun and Schwartz (2025) demonstrate, platform A/B testing tools optimize ad delivery based on predicted user-content relevance, meaning that the casual video and the professional video may be served to systematically different user populations. Gordon et al. (2019) reinforce this concern by showing that even with rich individual-level data, selection effects in digital advertising cannot be adequately controlled for after the fact. Additionally, a field experiment does not easily permit manipulation checks, so the researcher cannot directly verify that

the treatment operates through perceived authenticity rather than some other channel. Non-compliance is also a concern: not all users assigned to see an ad will actually be exposed, creating one-sided non-compliance that shifts the estimand from ATE to ITT or LATE.

Recommended Design: Field Experiment with Mitigation Strategies

Despite the internal validity threats, the proposed design is a field experiment conducted through a social media platform's advertising infrastructure. The core justification is that the research question is innately about how consumers respond to influencer content *in its natural environment*. Content style effects are likely moderated by the social media context itself; the feed, the social proof signals, the fleeting attention span of scrolling users[1], and stripping away that context in a lab would risk studying a phenomenon that does not transfer to the real world. As Gordon et al. (2019) note, the value of field experiments in digital advertising lies precisely in their ability to measure effects on real behavior in real markets.

To address the divergent delivery problem, several mitigation strategies can be employed. First, the brand can configure the platform's ad manager to use *identical audience targeting* for both conditions, restricting delivery to a pre-defined demographic segment (e.g., Instagram users aged 18–30 in a specific geographic region) and disabling algorithmic optimization of creative delivery where the platform allows it. Second, each content variant includes a unique tracked link (UTM-tagged or affiliate URL) to the product page, enabling clean attribution of click-through, add-to-cart, and conversion behavior to the specific condition that generated it. Third, the analysis should report both the intent-to-treat (ITT) estimate and, where exposure data is available, the local average treatment effect (LATE) to account for non-compliance. Fourth, demographic and behavioral covariates collected by the platform (age, gender, usage frequency) can be included in the regression to improve precision and check for balance across conditions. While these measures cannot fully eliminate the divergent delivery concern, they substantially reduce it and make the remaining threat transparent and quantifiable.

The sacrifice relative to a lab experiment is the inability to include manipulation checks or mediator scales. The researcher cannot directly verify that any observed difference operates through perceived authenticity. This limitation should motivate a complementary lab study as a follow-up to probe the mechanism once the field experiment establishes whether a practically meaningful effect exists in the real market.

2.4 Validity Analysis

2.4.1 Threats to Internal Validity

The most serious internal validity threat is that the platform's algorithm may serve the two ad variants to systematically different audiences, even when targeting parameters are held constant. As Braun and Schwartz (2025) document, this *divergent delivery* means that any observed difference in outcomes could reflect audience composition rather than content style. The mitigation strategies discussed above, restricting targeting, disabling creative optimization where possible, and checking covariate balance post hoc reduce, but do not fully eliminate this threat. Transparency about residual confounding is essential when interpreting the results.

In a field experiment, not all users assigned to see an ad will actually view it. Some will scroll past before the video loads, others may be served the ad but not attend to it. This one-sided non-compliance means the experiment estimates an intent-to-treat (ITT) effect rather than the average treatment effect on the treated (ATT). Where the platform provides impression-level data, an instrumental-variables approach can recover the local average treatment effect (LATE), but the ITT remains the most policy-relevant estimand for managerial decision-making, since it reflects what happens when the brand *deploys* a given content strategy.

Despite using the same influencer and product across conditions, the casual and professional videos inevitably differ on multiple dimensions, not only authenticity but also humor, perceived effort, visual aesthetics, and potentially video pacing. This means the treatment is not a clean manipulation of a single construct. Unlike

[1] Colloquially referred to as "brain rot" by Gen Z users themselves named Oxford's Word of the Year for 2024.

in a lab setting, the field experiment does not allow manipulation checks to diagnose which dimensions are driving the effect. This limitation is inherent to the field design and should be acknowledged: the experiment tests the *bundle* of features that constitute casual versus professional content, not authenticity in isolation. Including platform-provided demographic and behavioral covariates in the analysis can partially account for heterogeneous responses, but it cannot tell us *why* the treatment works.

If the same user follows the influencer organically and encounters both the casual and professional content (e.g., through the influencer's own feed alongside the paid ad), treatment contamination may occur. This risk can be reduced by running the two ad variants in non-overlapping time windows or geographic regions, though this introduces potential confounds from temporal or regional differences. Alternatively, the brand can coordinate with the influencer to ensure that only the paid ads are posted during the experimental window.

Behavioral outcomes such as conversion are binary and typically have low base rates in influencer campaigns, meaning the experiment requires a large sample to detect modest effects. A formal power analysis based on historical conversion rates for similar campaigns should be conducted prior to launch. Under-powering the experiment risks a Type II error that will incorrectly state that content style does not matter.

2.4.2 Threats to External Validity

The norms and expectations of content differ across platforms. Casual, "off-the-cuff" content is native to TikTok but may feel out of place on LinkedIn or even Instagram. The field experiment is conducted on a single platform, and results should not be generalized to other platforms without replication. However, running the experiment *on* a platform, rather than abstracting away the platform context in a lab, ensures that the estimated effect is valid for at least one real market environment.

The effect of content style likely depends on the product category. Casual, humorous content may be highly effective for "enjoyment-driven", low-involvement products (e.g., snacks, fashion accessories) but less so for utilitarian, high-involvement products (e.g., financial services, electronics). The proposed experiment tests a single product category, and the results should not be generalized beyond it without replication.

The effect may depend on the particular influencer's persona. An influencer whose brand is already casual may show a smaller treatment effect than one whose audience expects polished content. While using a single influencer controls for person-level confounds, it limits generalizability. Future replications with multiple influencers would strengthen external validity.

Social media trends evolve rapidly. What reads as "authentic" today may feel forced or faux[2] in six months as the casual aesthetic becomes mainstream. The field experiment captures a snapshot of consumer preferences at a particular moment, and its conclusions may have a limited shelf life.

2.5 Discussion and Conclusion

This report has proposed an experimental design to test whether casual, authentic-style influencer content causes higher purchase intent than professionally produced influencer content. The focal treatment effect concerns perceived authenticity of the content style, operationalized through two video conditions featuring the same influencer and the same product but differing in production quality, tone, and delivery.

The recommended design is a between-subjects field experiment conducted through a social media platform's advertising infrastructure. This choice prioritizes external validity and behavioral realism; a trade-off that is justified by the nature of the research question. Content style effects are deeply embedded in the social media context: the feed environment, the social proof cues, and the fleeting attention of scrolling users all shape how authenticity is perceived and whether it translates into action. A lab experiment, while offering superior control and the ability to probe mechanisms through manipulation checks, would strip away precisely the contextual factors that make the question business-relevant in the first place.

The field design is not without limitations. The divergent delivery problem documented by Braun and Schwartz (2025) means that platform algorithms may confound the treatment comparison by serving the

[2] Like Wendy's replies on 𝕏 (formerly Twitter)

two ad variants to systematically different audiences. Gordon et al. (2019) further demonstrate that observational corrections for such selection effects are unreliable. The mitigation strategies proposed like restricting targeting parameters, disabling creative optimization, and checking covariate balance, reduce but do not eliminate this threat. Furthermore, the compound treatment concern remains: without manipulation checks, the experiment cannot isolate *perceived authenticity* from the bundle of features that distinguish casual from professional content. These limitations should motivate a complementary lab study as a follow-up to probe the mechanism once the field experiment establishes whether a practically meaningful effect exists in the real market.

From a practical standpoint, the answer to this research question has direct implications for how brands allocate their influencer marketing budgets. If casual content proves more effective at driving real conversions, the irony is notable: the less brands invest in production, the more they may earn in return . . . at least among the younger consumers who increasingly dictate market trends.

3 Task 3: The Return of the Gen-Z Intern

In this task, we analyze a promotional campaign where treatment (receiving a discount) was *not* randomized. We have a dataset of 5,000 observations with 20 covariates and must estimate the causal effect of the discount on sales (in USD) using several methods, relying on the conditional independence assumption. All code is provided in Appendix A.2.

3.1 EDA

First off, we can start by doing some exploratory data analysis to see what we're dealing with:

```
> Shape: (5000, 22)
> Treatment distribution: d=0: 2532, d=1: 2468
> Treatment rate: 0.494
>
> Outcome summary by treatment group:
>     count     mean     std    min     25%     50%     75%     max
> d
> 0  2532.0  1587.37  437.91 -328.0  1296.0  1604.0  1889.5  2921.0
> 1  2468.0  1844.19  411.71  347.0  1563.0  1854.0  2132.0  2984.0
>
> Naive difference in means: 256.81
```

We can observe a nearly 50/50 split in the data, so we don't have to worry about any extreme imbalance in treatment assignment. The naive ATE is 256.81 USD, suggesting that the treated group has substantially higher sales on average. The covariance balance looks surprisingly good, as most differences are tiny. The largest are in $X16(-0.29)$, $X17(-0.19)$, $X18(-0.15)$. These are small but could still confound. The 256.81 USD may still be biased despite the decent balance, we'll learn if that's the case when doing actual estimation.

3.2 Regression Adjustment

We estimate the ATE using OLS regression with all 20 covariates and heteroskedasticity-robust standard errors (HC1):

```
> Naive OLS (no controls):   ATE = 256.81, SE = 12.02, p < 0.001
> Regression adjustment:     ATE =  15.15, SE =  5.79, p = 0.009
>                            95% CI: [3.81, 26.50]
>                            R-squared: 0.883
```

The first thing we (should) take notice of in the output is the massive confounding. The naive estimate drops to 15.15 USD after controlling for covariates! The covariates were heavily confounding the raw comparison. The R^2 is 0.882, telling us that the 20 covariates explain 88.2% of the sales variance, which is encouraging for the conditional independence assumption: the covariates are highly informative. The main drivers are $X14(-304)$, $X15(-134)$, $X7(+22)$, $X5(+36)$, and $X20(-27)$. The treatment effect is still significant at 15.15 USD ($p = 0.009$, CI: $[3.81, 26.50]$); the discount appears to have a modest positive causal effect.

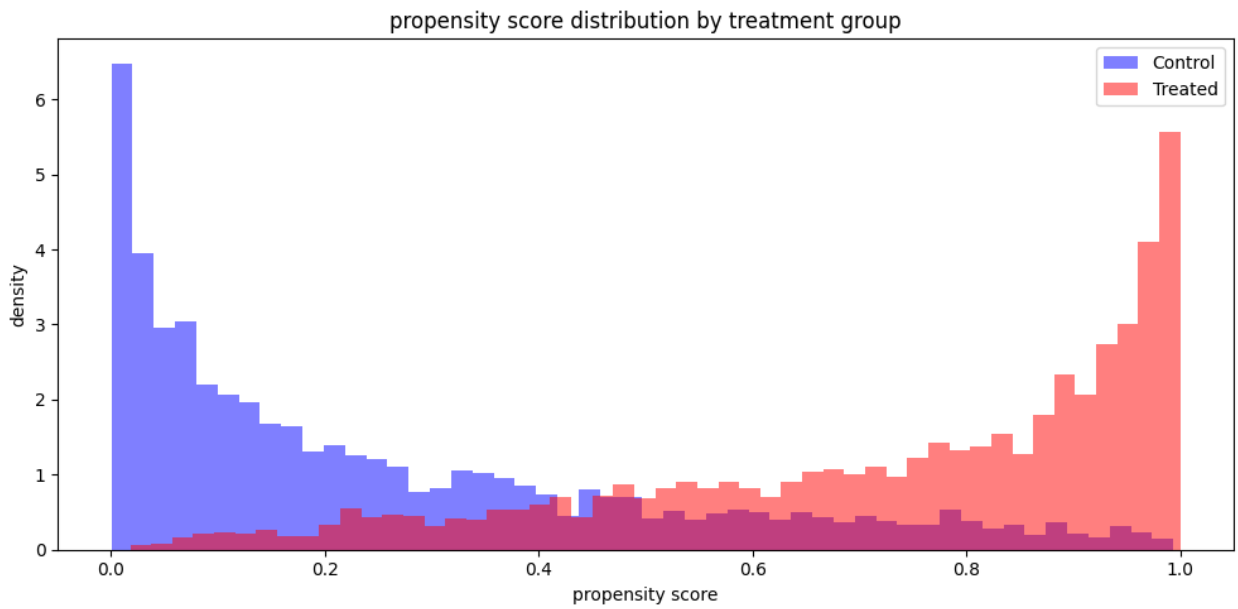
However, OLS regression adjustment has known limitations: it imposes a linear functional form, performs variance-weighted estimation, and does not protect against overfitting when the same sample is used for both fitting and inference.

3.3 Matching

3.3.1 Propensity Score Estimation and Common Support

We estimate propensity scores using logistic regression on standardized covariates:

```
> Propensity Score Summary:
> Control group - mean: 0.26, min: 0.0001, max: 0.99
> Treated group - mean: 0.73, min: 0.02,   max: 1.00
>
> Common support range: [0.02, 0.99]
> Observations in common support: 5000 / 5000 (100.0%)
```



The figure reveals that the propensity score distributions are heavily separated: control observations cluster near 0 and treated observations cluster near 1. This confirms that the covariates are highly predictive of treatment assignment, consistent with the strong confounding observed earlier. While technically all observations fall within the common support range, the density of the overlap in the middle range ($\sim 0.2-0.8$) is thin relative to the tails. This has direct implications for IPW (discussed below), where extreme propensity scores produce unstable weights.

3.3.2 Propensity Score Matching

We implement 1:1 nearest-neighbor matching on the propensity score (without replacement). The ATET is ~ 2.40, statistically insignificant (CI: $[\sim 23.70, 18.90]$). Match quality is excellent (mean propensity score distance of 0.0006), and covariance balance is reasonable (mean standardized mean difference of 0.075, max of 0.24).

```
> Propensity Score Matching (1:1 NN):
> ATET estimate: -2.40
> Std err: 10.87
> 95% CI: [-23.70, 18.90]
> Mean PS distance: 0.0006
> Max PS distance:  0.0064
```

3.3.3 Mahalanobis Distance Matching

As an alternative distance measure, we use Mahalanobis distance, which accounts for different variances and correlations between covariates. It is the most common alternative to propensity score matching for multivariate matching.

```
> Mahalanobis Distance Matching (1:1 NN):
> ATET estimate: 150.85
> Std err: 7.66
> 95% CI: [135.84, 165.87]
> Mean Mahalanobis distance: 3.45
> Max Mahalanobis distance:  5.37
```

The Mahalanobis estimate of 150.85 USD should not be trusted. Covariate balance diagnostics reveal a max standardized mean difference of 1.03, indicating at least one covariate is severely imbalanced after matching. This is the curse of dimensionality: with 20 covariates, Mahalanobis distance must simultaneously minimize discrepancies across all dimensions, resulting in poor matches overall. This illustrates perfectly why propensity score matching is preferred in high-dimensional settings like this one.

Method	**Mean |SMD|**	**Max |SMD|**
PS Matching	0.075	0.242
Mahalanobis	0.198	1.031

3.4 Inverse Probability Weighting (IPW)

Using the IPW estimator with propensity scores clipped to [0.01, 0.99] and bootstrap standard errors, we get an ATE of -311.69 USD:

```
> IPW Estimator:
> ATE estimate: -311.69
> Bootstrap SE: 81.21
> 95% CI: [-470.87, -152.51]
>
> Weight diagnostics:
> Treated weights - mean: 1.81, max: 56.29
> Control weights - mean: 17.44, max: 100.00
```

This is clearly unreliable. The weight diagnostics reveal the problem: the max control weight hits the clipping ceiling of 100 (1/0.01), and the mean control weight is 17.44, which is far from the ideal ~ 1. As the propensity score plot showed, many control observations have propensity scores near 0, producing extreme inverse weights that dominate the estimate.

3.5 Augmented Inverse Probability Weighting (AIPW)

The AIPW estimator combines propensity scores weighting with outcome regression, using a linear regression outcome model and logistic regression propensity model:

```
> AIPW Estimator (Linear + Logistic):
> ATE estimate: 18.85
> Bootstrap SE: 8.42
> 95% CI: [2.34, 35.35]
```

The ATE is 18.85 USD (SE = 8.42, CI: [2.34, 35.35]), close to the regression adjustment estimate (15.15). This is expected: when the outcome model is well-specified, the IPW correction term becomes small, and the estimator is primarily driven by the regression component. This is the double robustness property.

However, our AIPW implementation does not use cross-fitting, meaning the same observations are used to fit both nuisance models and estimate the treatment effect. This can introduce overfitting bias.

3.6 Double ML

DML addresses the two key challenges of using ML in causal inference: regularization bias (via orthogonalization, based on the Frisch-Waugh-Lovell theorem) and overfitting bias (via 5-fold cross-fitting). We explore three ML algorithms for both the outcome and treatment models:

```
> DML Summary:
> ML Method                    ATE      SE     95% CI
> ---------------------------------------------------------
> Lasso                      1.00    6.00    [-10.77, 12.77]
> Random Forest              1.00    5.83    [-10.42, 12.42]
> Gradient Boosting          1.00    5.94    [-10.65, 12.65]
```

All three converge to ATE $\sim$ 1.00 USD, which is statistically insignificant. The convergence across fundamentally different ML methods (linear regularized, ensemble bagging, ensemble boosting) provides strong evidence that the result is robust to model choice.

3.7 Comparison and Discussion

Method	ATE (USD)	Significant?	Trust level
Naive (no controls)	256.81	Yes	None (confounded)
Regression adj.	15.15	Yes ($p = 0.009$)	Moderate
PS Matching	-2.40	No	Moderate
Mahalanobis	150.85	Yes	Low (bad balance)
IPW	-311.69	Yes	None (extreme weights)
AIPW	18.85	Yes ($p < 0.05$)	Moderate-high
DML (Lasso)	1	No ($p = 0.87$)	High
DML (Random Forest)	1	No ($p = 0.87$)	High
DML (Gradient Boost)	1	No ($p = 0.87$)	High

3.7.1 Why do the estimates differ?

The estimates split into two groups: methods without cross-fitting (OLS and AIPW) suggest a modest significant effect of 15-19 USD, while methods with cross-fitting (DML) or nonparametric approaches (PS matching) find a near-zero, insignificant effect of -2 to 1 USD. This pattern is consistent with overfitting bias. DML's cross-fitting explicitly breaks the correlation between prediction errors and the outcome's random errors.

The Mahalanobis estimate (150.85 USD) is an outlier driven by poor matching quality in 20 dimensions, while IPW (-311.69 USD) is dominated by extreme inverse probability weights caused by the separated propensity score distributions.

3.7.2 Which estimates do we trust the most?

DML is the most methodologically rigorous estimator in our comparison:

- It addresses regularization bias through orthogonalization (the Frisch-Waugh-Lovell principle), ensuring that shrinkage in the ML models does not propagate into the treatment effect estimates.
- It addresses overfitting bias through 5-fold cross-fitting, ensuring that predictions are independent of the estimation sample.
- It allows non-linear confounding through flexible ML, relaxing the linearity assumption that OLS regression imposes.
- All three ML algorithms converge to the same estimate, demonstrating robustness to model choice.

The PS matching result (-2.40 USD, insignificant) corroborates the DML finding, despite using a completely different methodology.

3.7.3 Assumptions and limitations

All methods in this analysis rely on the *conditional independence assumptions*: the potential outcomes are independent of treatment conditional on the covariates. If there exist unobserved confounders that influence both treatment assignment and sales, all estimates are biased. The high R^2 (0.883) is reassuring; the 20 covariates explain the vast majority of outcome variation, but it does not guarantee that the selection mechanism is fully captured.

Additionally, the separated propensity score distributions indicate that the treated and control groups occupy somewhat different regions of the covariate space. While common support technically holds, the thin overlap in the middle range means that some estimates rely on comparisons between units that are not closely comparable, particularly for IPW which assigns extreme weights to observations in low-density regions.

3.7.4 Recommendation to the marketing manager

Based on our analysis, and particularly the DML estimates which we consider most trustworthy, the discount campaign likely has no meaningful causal effect on sales. The naive difference of 257 USD was almost entirely driven by selection effects: customers who received discounts were systematically different from those that did not, and these differences (and not the discount itself) explain the higher sales.

We recommend the manager not assume that discounts drive incremental revenue based on this observational data. To obtain a definitive answer, we recommend conducting a randomized experiment where discounts are assigned randomly to a subset of customers. This would eliminate confounding by design and provide an unbiased estimate of the discount's true effect. If randomization is infeasible, the company should at minimum collect and control for all variables that influence both discount targeting and purchasing behavior.

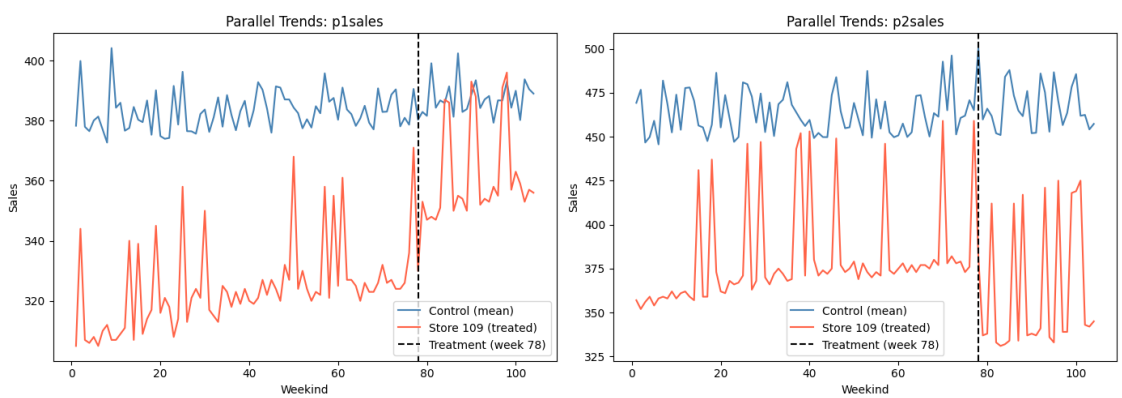
4 Task 4: The sales analyst

In this task, we evaluate the causal impact of a new in-store marketing strategy on weekly sales of two products at store 109, a single treated store in a multi-store retail chain. A randomized design was not feasible, so we rely on observational panel data covering 21 stores across 104 weeks (2024–2025). The treatment began in week 26 of 2025 (weekind 78). We apply five progressively more sophisticated causal inference methods: pre-post analysis, two-way fixed effects, difference-in-differences, synthetic control, and synthetic DID, and compare their estimates to assess the strategy's effectiveness. All code is provided in Appendix A.3.

4.1 Exploratory Data Analysis

We start off with some EDA:

```
>  -- p1sales --
>                 mean    std  count
> Treated Post
> 0       0     383.05  69.16   1540
>         1     387.32  82.97    540
> 1       0     324.36  14.40     77
>         1     360.85  17.05     27
>
> -- p2sales --
>                 mean     std  count
> Treated Post
> 0       0     463.18  112.56   1540
>         1     467.91  122.22    540
> 1       0     380.49   28.59     77
>         1     362.96   38.01     27
```



For product 1, we notice that store 109 has much lower sales than control stores (around 320 vs. just over 380 pre-treatment). The levels are very different. Pre-treatment trends look roughly parallel in the plot, both lines are relatively flat with similar fluctuating patterns. Post-treatment, store 109 jumps to around 360 (a roughly +40 unit increase), while control barely moves. This hints at a positive treatment effect for product 1

For product 2, store 109 is yet again below the control group ($\sim$ 375 vs. 460 pre-treatment). Post-treatment, store 109 *drops* to much lower levels, while control is flat. This suggests a possible negative effect on product 2 – or at least no positive effect. The pre-treatment trends are noisier, and the spikes in store 109 look like promotion-driven jumps, which is worth controlling for. The large level differences are fine for DID (which relies on trends, not levels), but they do motivate synthetic control as a potentially better approach.

4.2 Pre-Post Analysis (store 109 only)

This is the simplest analysis, where we just look at store 109 before vs. after, controlling for covariates. It is biased (no counterfactual), but it is a starting point.

```
> p1sales
> Formula: p1sales ~ post + p1price + p1prom + compind
>                Coef.  Std.Err.          z          P>|z|      [0.025  \
> Intercept  282.676750  5.631702  50.193843   0.000000e+00  271.638817
> post        33.689387  1.544682  21.809924  1.867844e-105   30.661867
> p1price     -0.470422  2.408546  -0.195313   8.451476e-01   -5.191086
> p1prom      35.286051  2.354490  14.986706   8.969257e-51   30.671335
> compind    103.459690  2.061203  50.193843   0.000000e+00   99.419807
>
>            0.975]
> Intercept  293.714682
> post        36.716907
> p1price      4.250243
> p1prom      39.900766
> compind    107.499574
>
> Pre-post treatment effect estimate: 33.69 (p=0.0000)
> p2sales
> Formula: p2sales ~ post + p2price + p2prom + compind
>                Coef.  Std.Err.          z         P>|z|      [0.025  \
> Intercept  335.448499  5.845090  57.389790  0.000000e+00  323.992332
> post       -29.710065  2.104538 -14.117145  2.978156e-45  -33.834883
> p2price     -4.068014  2.363620  -1.721094  8.523370e-02   -8.700624
> p2prom      78.255946  1.972232  39.678879  0.000000e+00   74.390443
> compind    122.774150  2.139303  57.389790  0.000000e+00  118.581194
>
>            0.975]
> Intercept  346.904665
> post       -25.585247
> p2price      0.564597
> p2prom      82.121449
> compind    126.967107
>
> Pre-post treatment effect estimate: -29.71 (p=0.0000)
```

The naive pre-post estimate shows a +33.7 unit increase in weekly sales after treatment (highly significant). Promotions also have a large effect (+35.3), while price has essentially no effect in this single-store sample. The competition index (`compind`) is large but that is because it is on a different scale (0-1).

The pre-post estimate shows a -29.7 unit decrease; the marketing strategy appears to have hurt product 2 sales. Promotions have a huge effect (+78.3) and price is marginally negative ($p = 0.085$).

There is one key caveat, and that is that this analysis has no counterfactual. Any time trend common to all stores is absored into the post coefficient. That is why we need DID and synthetic control.

4.3 Two-Way Fixed Effects Panel Model

This uses all stores with store and time fixed effects. The `treat\_post` coefficient is the treatment effect, controlling for anything constant within a store (store FE) and anything common across stores in a given week (time FE):

```
> -- p1sales --
> Variable              Coef   Std.Err.    P-value
> ----------------------------------------
> treat_post          29.979      4.256     0.0000
> p1price             -2.475      0.891     0.0055
> p1prom              37.590      1.470     0.0000
>
> TWFE treatment effect: 29.98 (p=0.0000)
> R-squared: 0.9773
> N = 2184
>
> -- p2sales --
> Variable              Coef   Std.Err.    P-value
> ----------------------------------------
> treat_post         -34.291      4.699     0.0000
> p2price             -2.951      0.861     0.0006
> p2prom              89.424      4.538     0.0000
>
> TWFE treatment effect: -34.29 (p=0.0000)
> R-squared: 0.9870
> N = 2184
```

For product 1, TWFE estimates a treatment effect of +29.98 units ($p < 0.001$). This is slightly smaller than the naive pre-post (+33.7), which makes sense because the TWFE controls for time-varying shocks common to all stores via week FE. Very high R^2 (0.977) means the FEs and covariates explain almost all variation. Price now shows a significant negative effect (-2.48, $p = 0.006$) since we have cross-store variation to identify it.

For product 2, TWFE estimates -34.29 units ($p < 0.001$). The marketing strategy appears to have *reduced* product 2 sales even after controlling for store and time effects. Promotions have a massive effect (+89.42), and price is significantly negative (-2.95).

With only 21 stores (clusters), cluster-robust SEs can be unreliable. We'll keep this in mind when comparing across methods. The story is consistent so far: positive for P1, negative for P2. Let us commence explicit DID framing.

4.4 Difference-in-Differences

First we run a simple 2×2 DID (collapsed means), then a panel DID regression:

```
> -- 3a: simple 2x2 did (raw means) --
>     p1sales:
>     Treated:  pre=324.36  post=360.85  diff=+36.49
>     Control:  pre=383.05   post=387.32   diff=+4.27
>     DID estimate: +32.22
>
>     p2sales:
>     Treated:  pre=380.49  post=362.96  diff=-17.53
>     Control:  pre=463.18   post=467.91   diff=+4.73
>     DID estimate: -22.26
>
>
> -- 3b: did regression (w covariates) --
>     p1sales
>     Variable              Coef    Std.Err    P-value
>     ----------------------------------------------
>     treated            -33.633      1.109     0.0000
>     post                 3.538      4.066     0.3842
>     treat_post          29.974      4.138     0.0000
>     p1price             -2.585      0.800     0.0012
>     p1prom              37.619      1.457     0.0000
>
>     DID treatment effect: 29.97 (p=0.0000)
>     R-squared: 0.9767, N = 2184
>
>     p2sales
>     Variable              Coef    Std.Err    P-value
>     ----------------------------------------------
>     treated            -18.371      1.136     0.0000
>     post                 2.905      4.167     0.4857
>     treat_post         -34.354      4.570     0.0000
>     p2price             -2.881      0.739     0.0001
>     p2prom              89.915      4.820     0.0000
>
>     DID treatment effect: -34.35 (p=0.0000)
>     R-squared: 0.9864, N = 2184
```

The simple 2×2 DID (raw means) shows a P1 increase of +32.22. Store 109 gained 36.5 units while control gained only 4.3, so the net treatment effect is +32.2. It shows a P2 decrease of -22.26, so store 109 dropped 17.5 while control gained 4.7, so the net effect is -22.3.

The DID regression with control variates shows a P1 increase of +29.97 ($p < 0.001$), which is very close to the TWFE estimate (29.98), as expected. P2 displays a decrease of -34.35 ($p < 0.001$), which is also very close to the TWFE (-34.29).

The `post` coefficient is small and insignificant in both regressions (3.5 and 2.9), meaning that there is no strong common time trend beyond what covariates capture, which is a good sign. The `treated` coefficient shows store 109 is structurally lower in sales (-33.6 for P1, -18.4 for P2).

Notice the raw 2×2 DID differs from the regression DID because the regression controls for prices and promotions. The regression version is more trustworthy.

One key concern for DID is that the parallel trends plot showed store 109 is at a very different level from

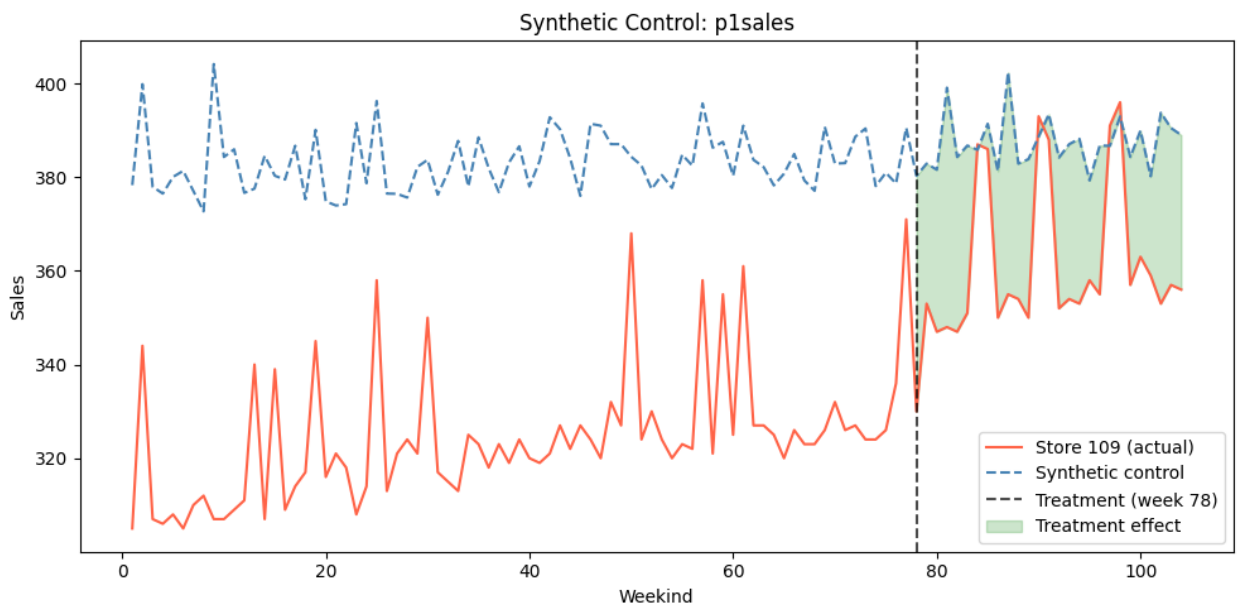
the control mean, and there are only 20 control stores being averaged equally. Some stores may be much better counterfactuals than others. This motivates the synthetic control approach.

4.5 Synthetic Control Method

This constructs a weighted combination of control stores that best matches store 109's pre-treatment sales trajectory. It is ideal here because we have 1 treated unit and a rich donor pool.

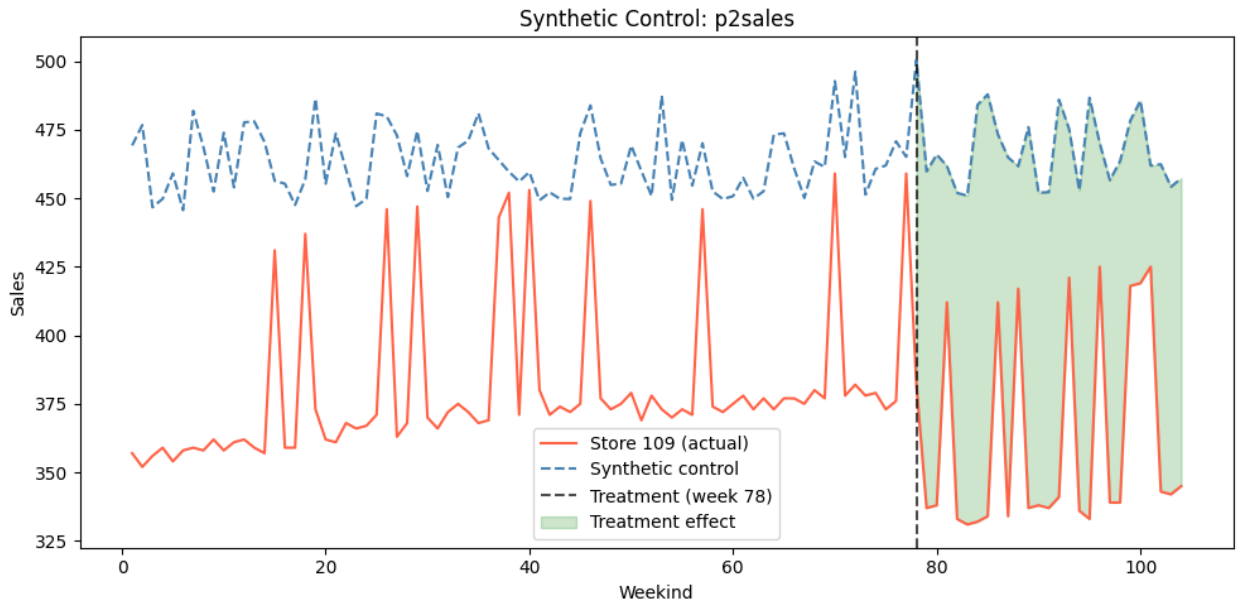
All stores get assigned a donor weight of 0.05 for both P1 and P2 sales. For P1 sales, we get the following results:

```
> Pre-treatment RMSE: 60.19
> ATT (avg post-treatment effect): -26.47
> Post-period effects by week: min=-51.1, max=4.3
```



And for P2 sales, we get these results:

```
> Pre-treatment RMSE: 87.43
> ATT (avg post-treatment effect): -104.94
> Post-period effects by week: min=-153.9, max=-36.9
```



The SCM completely failed here. Every donor got equal weight ($1/20 = 0.05$). The optimizer just returned the uniform average because it couldn't find *any* weighted combination of control stores that matches store 109's level. The pre-treatment RMSE is enormous (60 and 87), and the plots show the synthetic sits way above the actual.

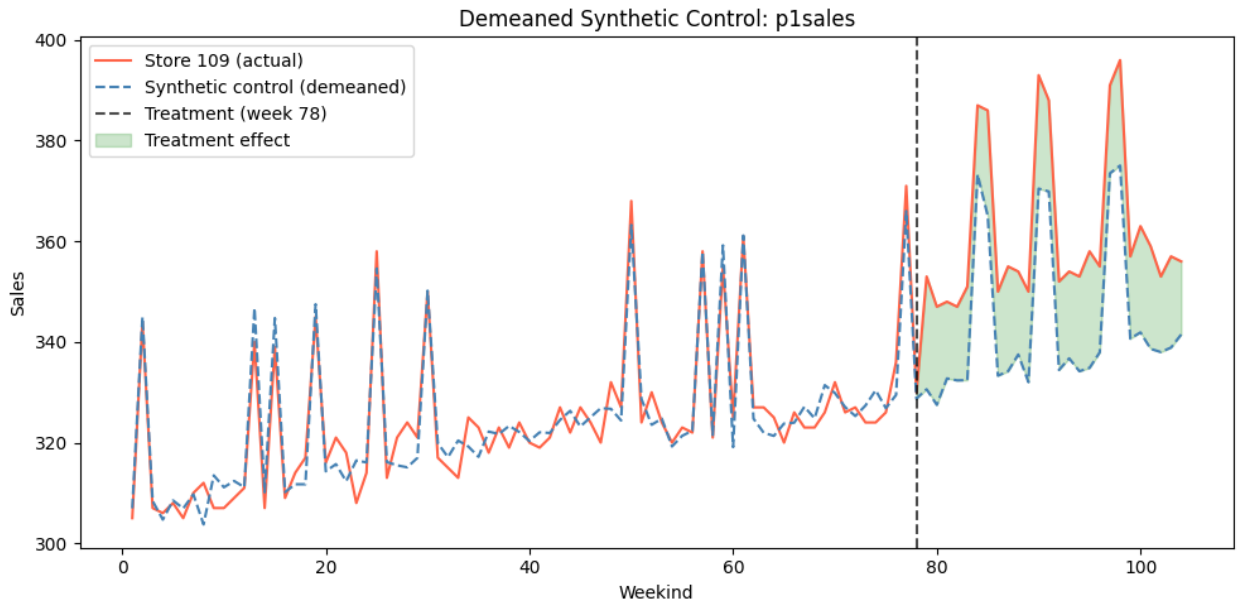
The reason why is because store 109 has much lower sales than *all* control stores. Since SCM weights are constrained to [0, 1] and sum to 1, the synthetic can never go below the lowest control store. Store 109 is outside the donor pool's range.

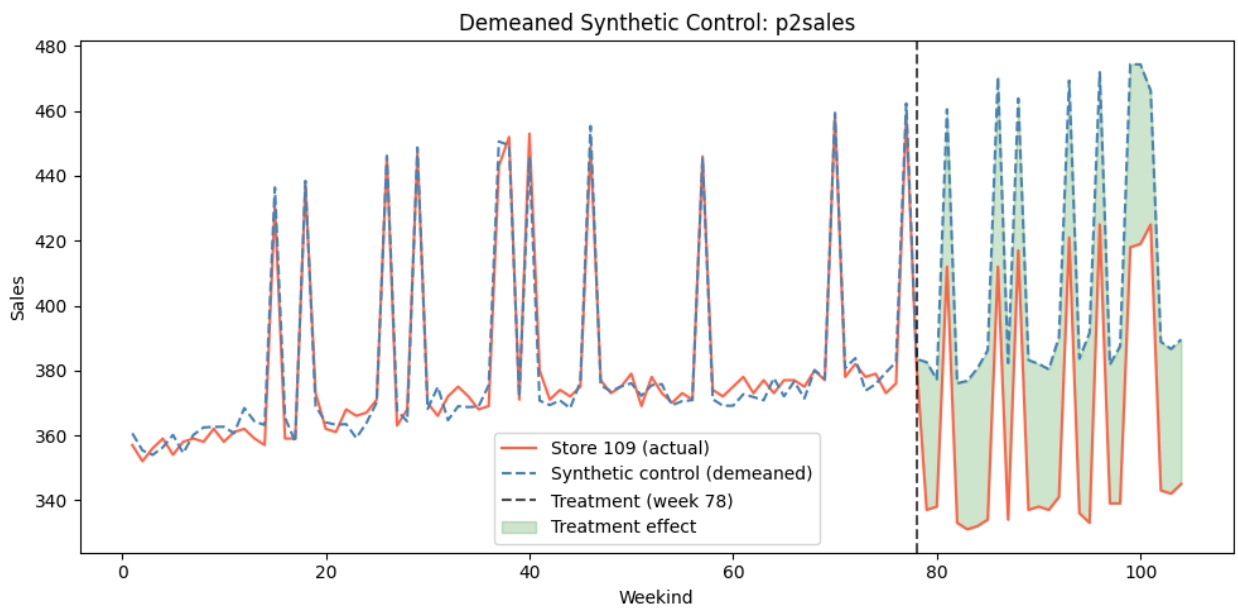
The fix here is a demeaned SCM, as discussed by Ferman and Pinto (2021). We subtract each store's pre-treatment mean, match on the *trajectories/patterns* instead of levels, then add back the treated store's mean:

4.6 Demeaned SCM

The demeaned SCM performs beautifully:

```
> ==================================================
> p1sales
> ==================================================
>
> Donor weights (non-zero, >1%):
>     Store 110: 0.4333
>     Store 111: 0.4332
>     Store 113: 0.0393
>     Store 114: 0.0231
>     Store 118: 0.0256
>     Store 119: 0.0394
>
> Pre-treatment RMSE: 4.05
> ATT (avg post-treatment effect): 17.66
> Post-period effects by week: min=1.3, max=23.2
>
> ==================================================
> p2sales
> ==================================================
>
> Donor weights (non-zero, >1%):
>     Store 103: 0.0254
>     Store 105: 0.0300
>     Store 110: 0.3510
>     Store 111: 0.4430
>     Store 114: 0.0175
>     Store 117: 0.0563
>     Store 119: 0.0177
>     Store 120: 0.0364
>     Store 121: 0.0138
>
> Pre-treatment RMSE: 4.13
> ATT (avg post-treatment effect): -46.00
> Post-period effects by week: min=-58.6, max=-1.5
```





The RMSE drops from 60/87 to 4.05/4.13, and the synthetic now tracks store 109 almost perfectly in the pre-period. The plots confirm this: the blue dashed line hugs the red line closely before week 78.

Both products lean heavily on stores 110 and 111 ($\sim$ 87% combined weight for P1, $\sim$ 79% for P2). These are the stores whose fluctuation patterns most resemble store 109. The remaining weight is spread thinly across a few others.

P1 has an ATT of +17.66, which is positive but notably smaller than DID (+30.0) and pre-post (+33.7). Every single post-treatment week shows a positive effect (min=+1.3), so the effect is consistent. The SCM suggests a more modest uplift.

P2 has an ATT of -46.00, which is even more negative than DID (-34.3). Every post-treatment week is negative (max=-1.5). The green gap in the plot is large and persistent.

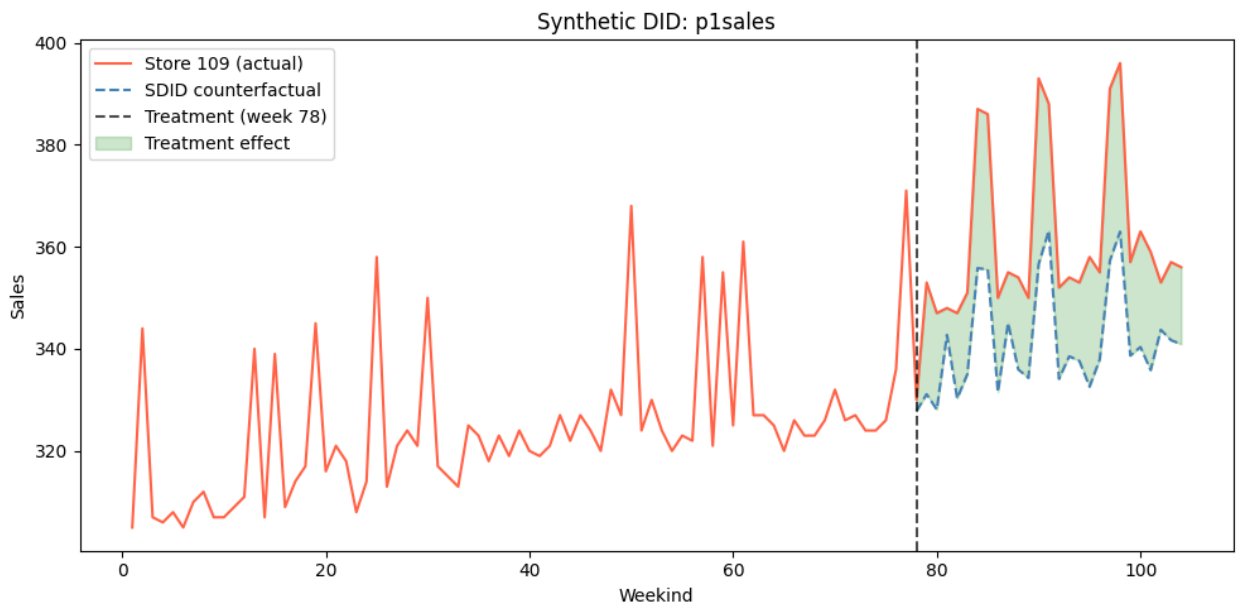
The reason why SCM gives different numbers than DID is because DID uses all 20 control stores equally (via the parallel trends assumption). SCM found that stores 110 and 111 are the best counterfactuals. If those stores had different trajectories from the average control store in the post-period, that explains the divergence. The SCM is likely more credible here because it builds a tailored counterfactual rather than assuming all stores trend identically.

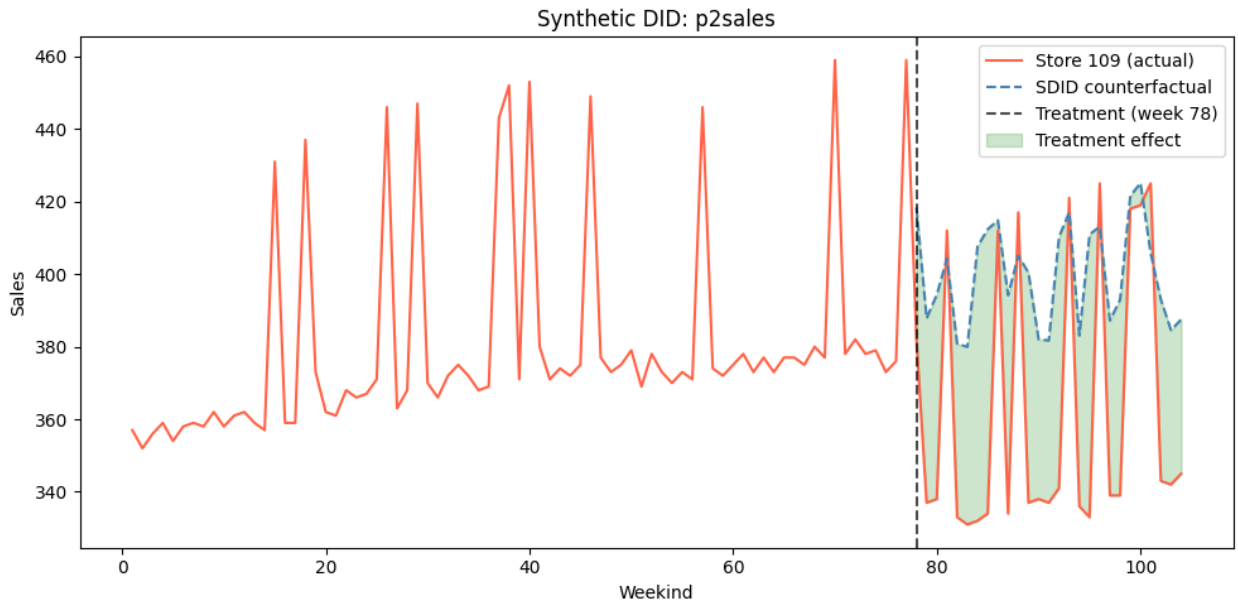
4.7 Synthetic DID

SDID (Arkhangelsky et al., 2021) combines the best of both worlds: it reweights both *units* (like SCM) and *time periods* (like DID). This makes it more robust when neither pure parallel trends nor pure SCM fit holds perfectly.

```
> p1sales
> sigma_hat: 22.81, zeta: 51.98
> omega_0: -85.68
> donor weights >1%:
>     store 101: 0.0309
>     store 102: 0.0309
>     store 103: 0.0309
>     store 104: 0.0309
>     store 105: 0.0311
>     store 106: 0.0309
>     store 107: 0.0281
>     store 108: 0.0281
>     store 110: 0.0337
>     store 111: 0.4667
>     store 112: 0.0281
>     store 113: 0.0313
>     store 114: 0.0281
>     store 115: 0.0281
>     store 116: 0.0281
>     store 117: 0.0281
>     store 118: 0.0309
>     store 119: 0.0309
>     store 120: 0.0294
>     store 121: 0.0281
> lambda_0: 1.96
> non-uniform time weights: 19 of 77
> last 10 pre-period weight: 0.3840
> delta_treated: 27.27, delta_control: 7.73
> tau: 19.54

> p2sales
> sigma_hat: 48.99, zeta: 111.68
> omega_0: -87.17
> donor weights >1%:
>     store 101: 0.0383
>     store 102: 0.0350
>     store 103: 0.0429
>     store 104: 0.0411
>     store 105: 0.0390
>     store 106: 0.0389
>     store 107: 0.0334
>     store 108: 0.0366
>     store 110: 0.1065
```

```
>     store 111: 0.1213
>     store 112: 0.0444
>     store 113: 0.0496
>     store 114: 0.0467
>     store 115: 0.0437
>     store 116: 0.0429
>     store 117: 0.0447
>     store 118: 0.0433
>     store 119: 0.0594
>     store 120: 0.0546
>     store 121: 0.0376
> lambda_0: 4.33
> non-uniform time weights: 4 of 77
> last 10 pre-period weight: 0.2331
> delta_treated: -29.89, delta_control: 7.00
> tau: -36.89
```





The intercept is doing its job: `omega_0 = -85.68` (P1) and `-87.17` (P2) absorbs the $\sim 60 - 80$ unit level gap between store 109 and the control stores.

For P1 unit weights, store 111 dominates at 0.47, with the rest at around 0.03. This is somehwat concentrated but less extreme than the demeaned SCM (which had stores 110 and 111 at 0.87 combined). The regularization intentionally spreads weight more evenly, and this is by design in SDID: as `zeta` grows, SDID interpolates between SCM (concentrated weights) and DID (uniform weights).

The P2 unit weights are all nearly uniform (0.03 - 0.12). This is because `sigma_hat` is much higher for P2 (48.99 vs. 22.81) due to those large promotion-driven spikes, which inflates `zeta` to 111.68. High regularization pushes SDID towards DID. This is actually the correct behavior; when the outcome series is noisy, SDID trusts the DID averaging more than the SCM matching.

For P1, 38% of the weights is on the last 10 pre-periods, meaning SDID upweights recent history. For P2, only 23%, which again is more regularized due to noise.

The demeaned SCM implementation produced the best pre-treatment fit (RMSE ~ 4). However, one thing worth noting is that we matched only on *outcome trajectories*, not on covariates. The classical Abadie et al. approach also matches on predictor variables. Since our fit is already excellent, adding covariates would not change much.

4.8 Which methods should we trust most?

The pre-post analysis (Section 4.1) is the weakest. There is no counterfactual, so any common time trend gets baked into the estimate. It is useful only as a baseline. DID/TWFE (Sections 4.2 and 4.3) are the workhorses. The DID regression and TWFE give nearly identical estimates ($\sim +30/-34$) because they are algebraically equivalent when you include store + time FE. The parallel trends assumption is somewhat supported by the pre-treatment plots (both P1 and P2 show roughly parallel movement), but the *level gap* between store 109 and the control mean is large. DID assumes all 20 control stores contribute equally to the counterfactual, which is crude.

The standard SCM (Section 4.4) failed due to the convex hull problem, but the demeaned SCM (Section 4.5) is arguably the most credible method for this setting. With only 1 treated unit, SCM's strength, which is building a tailored counterfactual from the best-matching donors, is exactly what we need. The pre-treatment fit was excellent (RMSE ~ 4), and the weights concentrated on stores 110 and 111, which track

store 109's fluctuation patterns closely. The lower estimates (+18 for P1, ~ -46 for P2) suggest that the DID methods may be biased because the average control store is not a good counterfactual for store 109.

SDID (Section 4.6) sits between DID and SCM by design. For P1, the weights concentrated on store 111 (similar to SCM), giving +19.5. For P2, high noise from promotion spikes inflated the regularization, pushing weights toward uniform (i.e., towards DID), giving -36.9. SDID's advantage is its double-robustness: it is consistent if *either* the parallel trends or the SCM fit holds.

Therefore we would recommend emphasizing the SCM and SDID estimates as the primary findings, while keeping DID as a robustness check. The SCM pre-treatment fit provides strong visual evidence of counterfactual validity.

There are some challenges and limitations to discuss. There is only one single treated unit, where all inference is based on one store. There is no way to assess treatment effect heterogeneity across stores. There are only 21 clusters for cluster-robust SEs in DID. Standard errors may be unreliable. SCM has no standard inference; we could run placebo tests (apply SCM to each control store pretending it's treated), but we did not. The post-period is relatively short at just 27 weeks, so it is hard to know if the effect is actually persistent. The promotion spikes in P2 create noise that complicates SCM matching and inflates SDID regularization. There are no spillovers assumed, so if the marketing strategy in store 109 affected nearby stores, the control group is contaminated, which is a SUTVA violation. Finally, store 109 is an outlier in sales levels. It is already well below all control stores, raising questions about external validity.

Our recommendations to management, using the SCM/SDID estimates (which we argue are the most credible), is that the strategy lifts P1 sales by approximately +18 to +20 units per week. However, it also reduces P2 sales by approximately -37 to -46 units per week, so the net effect is negative. Before rolling out chain-wide, management should investigate *why* P2 declines and whether the strategy can be modified to avoid cannibalization. A proper randomized trial across multiple stores would provide definitive evidence with valid statistical inference.

References

Arkhangelsky, D., Athey, S., Hirshberg, D. A., Imbens, G. W., & Wager, S. (2021). Synthetic difference-in-differences. *American Economic Review*, *111*(12), 4088–4118. https://doi.org/10.1257/aer.20190159

Balaskas, S., Kotsari, G., & Rigou, M. (2024). Perspectives of young digital natives on digital marketing: Exploring annoyance and effectiveness with eye-tracking analysis. *Future Internet*, *16*(4). https://doi.org/10.3390/fi16040125

Braun, M., & Schwartz, E. M. (2025). Where A/B testing goes wrong: How divergent delivery affects what online experiments cannot (and can) tell you about how customers respond to advertising. *Journal of Marketing*, *89*(2), 71–95. https://doi.org/10.1177/00222429241275886

Djafarova, E., & Rushworth, C. (2017). Exploring the credibility of online celebrities' instagram profiles in influencing the purchase decisions of young female users. *Computers in Human Behavior*, *68*, 1–7. https://doi.org/https://doi.org/10.1016/j.chb.2016.11.009

Ferman, B., & Pinto, C. (2021). Synthetic controls with imperfect pretreatment fit. *Quantitative Economics*, *12*(4), 1197–1221. https://doi.org/10.3982/QE1596

Gohil, S. A., Nagariya, B. K., Parmar, A. U., Joshi, P. D., & Joshi, D. B. (2025). The impact of influencer authenticity on purchase intentions among Gen Z consumers. *Advances in Consumer Research*, *2*(5), 890–903. https://acr-journal.com/article/the-impact-of-influencer-authenticity-on-purchase-intentions-among-gen-z-consumers--1767/

Gordon, B. R., Zettelmeyer, F., Bhargava, N., & Chapsky, D. (2019). A comparison of approaches to advertising measurement: Evidence from big field experiments at Facebook. *Marketing Science*, *38*(2), 193–225. https://doi.org/10.1287/mksc.2018.1135

Singer, M. F., Callendar, C. L., Ma, X., & Tham, S. M. (2023). Differences in perceived influencer authenticity: A comparison of Gen Z and Millennials' definitions of influencer authenticity during the de-influencer movement. *Online Media and Global Communication*, *2*(3), 351–378. https://doi.org/10.1515/omgc-2023-0038

Sterling, G. (2015, August). Ad-blocking report: Nearly 200 million users, $22 billion in lost ad revenue [Accessed: 2026-02-12]. https://martech.org/ad-blocking-report-nearly-200-million-users-22-billion-in-lost-ad-revenue/

A Code

A.1 Task 1 Code

```python
import pandas as pd
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
from linearmodels.iv import IV2SLS

pd.set_option('display.max_columns', None)
df = pd.read_csv('CaseData2026.csv', delimiter=';')

treatment_group = df[df['test'] == 1]
control_group = df[df['test'] == 0]

treatment_no_exposure = treatment_group[treatment_group['impressions'] == 0]
treatment_with_exposure = treatment_group[treatment_group['impressions'] > 0]

compliance_rate_treatment = len(treatment_with_exposure) / len(treatment_group)
non_compliance_rate = len(treatment_no_exposure) / len(treatment_group)

print(f"treatment group size: {len(treatment_group)}")
print(f"treatment group with impressions > 0 (compliers): {len(treatment_with_exposure)}")
print(f"treatment group with impressions = 0 (non-compliers): {len(treatment_no_exposure)}")
print(f"compliance rate in treatment: {compliance_rate_treatment:.4f} ({compliance_rate_treatment*100:.2f}%)")
print(f"non-compliance rate: {non_compliance_rate:.4f} ({non_compliance_rate*100:.2f}%)")


control_no_exposure = control_group[control_group['impressions'] == 0]
print(f'control group with 0 ghost impressions: {len(control_no_exposure) / len(control_group)}')
```

```
> treatment group size: 25072
> treatment group with impressions > 0 (compliers): 19694
> treatment group with impressions = 0 (non-compliers): 5378
> compliance rate in treatment: 0.7855 (78.55%)
> non-compliance rate: 0.2145 (21.45%)
> control group with 0 ghost impressions: 0.2159017971758665
```

```python
#bin var treatment indicator (actually saw ads)
df['saw_ads'] = ((df['test'] == 1) & (df['impressions'] > 0)).astype(int)

#ITT
itt_control = df[df['test'] == 0]['purchase'].mean()
itt_treatment = df[df['test'] == 1]['purchase'].mean()
itt_effect = itt_treatment - itt_control

print(f'control group conversion rate: {itt_control}')
print(f'treatment group conversion rate: {itt_treatment}\nITT effect: {itt_effect}
    ')

#ITT with regression (for st err)
X_itt = sm.add_constant(df['test'])
model_itt = sm.OLS(df['purchase'], X_itt).fit(cov_type='HC1') #robust SEs
print(f'regression:\n{model_itt.summary().tables[1]}')
```

```
> control group conversion rate: 0.024029204107830552
> treatment group conversion rate: 0.028677409061901724
> ITT effect: 0.004648204954071172
> regression:
> ==============================================================================
>                 coef    std err          z      P>|z|      [0.025      0.975]
> ------------------------------------------------------------------------------
> const          0.0240      0.001     24.773      0.000       0.022       0.026
> test           0.0046      0.001      3.245      0.001       0.002       0.007
> ==============================================================================
```

```python
#LATE via IV
#instrument: test, endo: saw_ads, outcome: purchase

df_iv = df[['purchase', 'saw_ads', 'test']].copy()
df_iv['const'] = 1

iv_model = IV2SLS(
    dependent=df_iv['purchase'],
    exog=df_iv['const'],
    endog=df_iv['saw_ads'],
    instruments=df_iv['test'],
).fit(cov_type='robust')

iv_model.summary
```

```
> IV-2SLS Estimation Summary
> Dep. Variable: purchase R-squared: 0.0015
> Estimator: IV-2SLS Adj. R-squared: 0.0015
> No. Observations: 50000 F-statistic: 10.544
> Date: Wed, Feb 04 2026 P-value (F-stat) 0.0012
> Time: 17:58:29 Distribution: chi2(1)
> Cov. Estimator: robust
> Parameter Estimates
> Parameter Std. Err. T-stat P-value Lower CI Upper CI
> const 0.0240 0.0010 24.774 0.0000 0.0221 0.0259
> saw_ads 0.0059 0.0018 3.2471 0.0012 0.0023 0.0095
```

```python
#manual LATE calc: LATE = ITT / compliance rate
compliance_diff = df[df['test'] == 1]['saw_ads'].mean() - df[df['test'] == 0]['saw_ads'].mean()
late = itt_effect / compliance_diff

print(f'itt effect: {itt_effect}\ncompliance diff: {compliance_diff}\nLATE: {late}')
```

```
> itt effect: 0.004648204954071172
> compliance diff: 0.7854977664326739
> LATE: 0.005917527907406948
```

```python
#create impression bins for strat analysis
df['imp_bins'] = pd.cut(df['impressions'], bins=[-1, 0, 1, 2, 3, 5, 10, 20], labels=['0', '1', '2', '3', '4-5', '6-10', '11+'])

#calc ITT bu imp bin
cate_by_bin = df.groupby(['imp_bins', 'test'])['purchase'].agg(['mean', 'count', 'std']).unstack()
cate_by_bin.columns = ['_'.join(map(str, col)) for col in cate_by_bin.columns] # convert to str before joining
cate_by_bin['ITT'] = cate_by_bin['mean_1'] - cate_by_bin['mean_0']
cate_by_bin['n_total'] = cate_by_bin['count_0'] + cate_by_bin['count_1']

cate_by_bin[['mean_0', 'mean_1', 'ITT', 'n_total']].round(4)
```

```
>     mean_0 mean_1 ITT n_total
> imp_bins
> 0 0.0000 0.0000 0.0000 10760
> 1 0.0000 0.0000 0.0000 2932
> 2 0.0000 0.0000 0.0000 5860
> 3 0.0000 0.0000 0.0000 7830
> 4-5 0.0000 0.0001 0.0001 14039
> 6-10 0.1302 0.1564 0.0262 8478
> 11+ 1.0000 1.0000 0.0000 101
```

```python
#regression with interaction to test heterogeneity
#model: purchase ~ test + impressions + test*impressions

df['test_x_imp'] = df['test'] * df['impressions']
X_interact = sm.add_constant(df[['test', 'impressions', 'test_x_imp']])
model_interact = sm.OLS(df['purchase'], X_interact).fit(cov_type='HC1')

print(model_interact.summary().tables[1])

print(f"- base effect of treatment (at 0 impressions): {model_interact.params['test']}")
print(f"- additional effect per impression: {model_interact.params['test_x_imp']}")
print(f"- interaction significant? p = {model_interact.pvalues['test_x_imp']}")
```

```
> ==============================================================================
>                  coef    std err          z      P>|z|      [0.025      0.975]
> ------------------------------------------------------------------------------
> const         -0.0533      0.002    -27.571      0.000      -0.057      -0.050
> test          -0.0067      0.003     -2.420      0.016      -0.012      -0.001
> impressions    0.0242      0.001     28.343      0.000       0.023       0.026
> test_x_imp     0.0035      0.001      2.866      0.004       0.001       0.006
> ==============================================================================
> - base effect of treatment (at 0 impressions): -0.0066651281558449925
> - additional effect per impression: 0.003506070910565647
> - interaction significant? p = 0.004154874372134771
```

```python
contribution_per_purchase = 200
cost_per_impression = 100 / 1_000

late_effect = 0.0059
itt_effect = 0.00465

avg_impressions_treatment = df[df['test'] == 1]['impressions'].mean()
avg_impressions_control = df[df['test'] == 0]['impressions'].mean()  # ghost impressions
avg_impressions_compliers = df[(df['test'] == 1) & (df['impressions'] > 0)]['impressions'].mean()

print(f"contribution per purchase: {contribution_per_purchase} NOK")
print(f"cost per impression: {cost_per_impression} NOK")
print(f"average impressions (treatment): {avg_impressions_treatment:.2f}")
print(f"average impressions (compliers only): {avg_impressions_compliers:.2f}")
```

```
> contribution per purchase: 200 NOK
> cost per impression: 0.1 NOK
> average impressions (treatment): 3.20
> average impressions (compliers only): 4.07
```

```python
# method 1: per-user analysis (ITT_based, conservative)

revenue_lift_per_user = itt_effect * contribution_per_purchase
cost_per_user = avg_impressions_treatment * cost_per_impression
net_contribution_per_user_itt = revenue_lift_per_user - cost_per_user

print(f'revenue lift per user: {revenue_lift_per_user}')
print(f'ad cost per user: {cost_per_user}')
print(f'net contribution per user: {net_contribution_per_user_itt}')
print(f'ROI: {(revenue_lift_per_user / cost_per_user - 1) * 100}')
```

```
> revenue lift per user: 0.9299999999999999
> ad cost per user: 0.32007418634333124
> net contribution per user: 0.6099258136566688
> ROI: 190.5576393475308
```

```python
# method 2: per-complier analysis (LATE-based)

revenue_lift_per_complier = late_effect * contribution_per_purchase
cost_per_complier = avg_impressions_compliers * cost_per_impression
net_contribution_per_complier = revenue_lift_per_complier - cost_per_complier

print(f"revenue lift per complier: {revenue_lift_per_complier} NOK")
print(f"ad cost per complier: {cost_per_complier} NOK")
print(f"net contribution per complier: {net_contribution_per_complier} NOK")
print(f"ROI: {(revenue_lift_per_complier / cost_per_complier - 1) * 100}%")
```

```
> revenue lift per complier: 1.18 NOK
> ad cost per complier: 0.4074794353610237 NOK
> net contribution per complier: 0.7725205646389762 NOK
> ROI: 189.58516617029494%
```

```python
# campaign-level calc

n_treatment = len(df[df['test'] == 1])
total_impressions_treatment = df[df['test'] == 1]['impressions'].sum()

total_revenue_lift = n_treatment * itt_effect * contribution_per_purchase
total_ad_cost = total_impressions_treatment * cost_per_impression
total_net_contribution = total_revenue_lift - total_ad_cost

print(f"users in treatment: {n_treatment}")
print(f"total impressions served: {total_impressions_treatment}")
print(f"expected additional conversions: {n_treatment * itt_effect}")
print(f"total revenue lift: {total_revenue_lift} NOK")
print(f"total ad cost: {total_ad_cost} NOK")
print(f"net contribution: {total_net_contribution} NOK")
```

```
> users in treatment: 25072
> total impressions served: 80249
> expected additional conversions: 116.58479999999999
> total revenue lift: 23316.96 NOK
> total ad cost: 8024.900000000001 NOK
> net contribution: 15292.059999999998 NOK
```

```python
# sensitivity analysis

#break-even contribution per purchase
breakeven_contribution = cost_per_user / itt_effect
print(f"break-even contribution per purchase:")
print(f"current: {contribution_per_purchase} NOK")
print(f"break-even: {breakeven_contribution} NOK")
print(f"margin of safety: {(contribution_per_purchase - breakeven_contribution) / contribution_per_purchase * 100}%")

#break-even ad cost per 1000 impressions
breakeven_cpm = (itt_effect * contribution_per_purchase) / avg_impressions_treatment * 1000
print(f"\nbreak-even CPM (cost per 1000 impressions):")
print(f"current: 100 NOK")
print(f"break-even: {breakeven_cpm} NOK")
print(f"margin of safety: {(breakeven_cpm - 100) / breakeven_cpm * 100}%")

#break-even treatment effect
breakeven_effect = cost_per_user / contribution_per_purchase
print(f"\nbreak-even treatment effect (ITT):")
print(f"current: {itt_effect*100}%")
print(f"break-even: {breakeven_effect*100}%")
print(f"margin of safety: {(itt_effect - breakeven_effect) / itt_effect * 100}%")
```

```
> break-even contribution per purchase:
> current: 200 NOK
> break-even: 68.83315835340457 NOK
> margin of safety: 65.58342082329771%
>
> break-even CPM (cost per 1000 impressions):
> current: 100 NOK
> break-even: 290.55763934753077 NOK
> margin of safety: 65.58342082329771%
>
> break-even treatment effect (ITT):
> current: 0.46499999999999997%
> break-even: 0.16003709317166562%
> margin of safety: 65.58342082329771%
```

```python
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

#sensitivity to contribution per purchase
contributions = np.linspace(50, 400, 100)
net_by_contrib = [itt_effect * c - cost_per_user for c in contributions]
axes[0].plot(contributions, net_by_contrib, 'b-', linewidth=2)
axes[0].axhline(y=0, color='red', linestyle='--', alpha=0.7)
axes[0].axvline(x=contribution_per_purchase, color='green', linestyle='--', alpha=0.7, label='Current (200)')
axes[0].axvline(x=breakeven_contribution, color='red', linestyle=':', alpha=0.7, label=f'Break-even ({breakeven_contribution:.0f})')
axes[0].set_xlabel('contribution per purchase (NOK)')
axes[0].set_ylabel('net contribution per user (NOK)')
axes[0].set_title('sensitivity to contribution')
axes[0].legend()
axes[0].fill_between(contributions, net_by_contrib, 0, where=[n > 0 for n in net_by_contrib], alpha=0.3, color='green')
axes[0].fill_between(contributions, net_by_contrib, 0, where=[n < 0 for n in net_by_contrib], alpha=0.3, color='red')

#sensitivity to CPM
cpms = np.linspace(0, 500, 100)
net_by_cpm = [itt_effect * contribution_per_purchase - avg_impressions_treatment * (c/1000) for c in cpms]
axes[1].plot(cpms, net_by_cpm, 'b-', linewidth=2)
axes[1].axhline(y=0, color='red', linestyle='--', alpha=0.7)
axes[1].axvline(x=100, color='green', linestyle='--', alpha=0.7, label='Current (100)')
axes[1].axvline(x=breakeven_cpm, color='red', linestyle=':', alpha=0.7, label=f'Break-even ({breakeven_cpm:.0f})')
axes[1].set_xlabel('CPM (NOK per 1000 impressions)')
axes[1].set_ylabel('net contribution per user (NOK)')
axes[1].set_title('sensitivity to ad cost')
axes[1].legend()
axes[1].fill_between(cpms, net_by_cpm, 0, where=[n > 0 for n in net_by_cpm], alpha=0.3, color='green')
axes[1].fill_between(cpms, net_by_cpm, 0, where=[n < 0 for n in net_by_cpm], alpha=0.3, color='red')

#sensitivity to treatment effect
effects = np.linspace(0, 0.02, 100)
net_by_effect = [e * contribution_per_purchase - cost_per_user for e in effects]
axes[2].plot(effects * 100, net_by_effect, 'b-', linewidth=2)
axes[2].axhline(y=0, color='red', linestyle='--', alpha=0.7)
axes[2].axvline(x=itt_effect*100, color='green', linestyle='--', alpha=0.7, label=f'Current ({itt_effect*100:.2f}%)')
axes[2].axvline(x=breakeven_effect*100, color='red', linestyle=':', alpha=0.7, label=f'Break-even ({breakeven_effect*100:.2f}%)')
axes[2].set_xlabel('treatment effect ITT (%)')
axes[2].set_ylabel('net contribution per user (NOK)')
axes[2].set_title('sensitivity to ad effectiveness')
axes[2].legend()
axes[2].fill_between(effects * 100, net_by_effect, 0, where=[n > 0 for n in net_by_effect], alpha=0.3, color='green')
axes[2].fill_between(effects * 100, net_by_effect, 0, where=[n < 0 for n in net_by_effect], alpha=0.3, color='red')
plt.tight_layout()
plt.show()
```

A.2 Task 3 Code

```python
import pandas as pd
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', None)
df = pd.read_csv('t3.csv')
```

```python
print(f'shape: {df.shape}')
print(f'\ntreatment distribution:\n{df['d'].value_counts()}')
print(f'\ntreatment rate: {df['d'].mean():.3f}')

print(f'\noutcome summary by treatment group')
print(df.groupby('d')['y'].describe().round(2))

print(f'\nnaive difference in means: {df[df['d']==1]['y'].mean() - df[df['d']==0]['y'].mean():.2f}')

print(f'\ncovariate balance (mean by treatment group)')
covariates = [f'X{i}' for i in range(1, 21)]
balance = df.groupby('d')[covariates].mean().T
balance.columns = ['Control (d=0)', 'Treated (d=1)']
balance['Diff'] = balance['Treated (d=1)'] - balance['Control (d=0)']
print(balance.round(3))
```

```
> shape: (5000, 22)

> treatment distribution:
> d
> 0    2532
> 1    2468
> Name: count, dtype: int64
>
> treatment rate: 0.494
>
> outcome summary by treatment group
>     count     mean     std    min     25%     50%     75%     max
> d
> 0  2532.0  1587.37  437.91 -328.0  1296.0  1604.0  1889.5  2921.0
> 1  2468.0  1844.19  411.71  347.0  1563.0  1854.0  2132.0  2984.0
>
> naive difference in means: 256.81
>
> covariate balance (mean by treatment group)
>      Control (d=0)  Treated (d=1)   Diff
> X1          24.065         24.059 -0.005
> X2          14.095         14.110  0.015
> X3          22.001         22.010  0.009
> X4          15.763         15.784  0.022
> X5           0.534          0.542  0.008
> X6          23.059         23.088  0.029
> X7          11.346         11.439  0.093
> X8          10.226         10.459  0.233
```

```
> X9          20.425         20.106 -0.319
> X10         15.643         14.784 -0.859
> X11         14.512         13.728 -0.784
> X12          6.941          5.728 -1.213
> X13         14.797         13.940 -0.857
> X14          5.536          4.943 -0.593
> X15         11.870         11.466 -0.404
> X16         11.614         11.321 -0.292
> X17         12.579         12.386 -0.193
> X18          8.018          7.868 -0.150
> X19         11.584         11.504 -0.080
> X20          0.347          0.346 -0.000
```

```python
covariates = [f'X{i}' for i in range(1, 21)]

X_naive = sm.add_constant(df[['d']])
model_naive = sm.OLS(df['y'], X_naive).fit(cov_type='HC1')

print('naive OLS (no controls)')
print(f'ATE estimate: {model_naive.params['d']:.2f}')
print(f'std err: {model_naive.bse['d']:.2f}')
print(f'p-value: {model_naive.pvalues['d']:.4f}')
print(f'95% CI: [{model_naive.conf_int().loc['d'][0]:.2f}, {model_naive.conf_int().loc['d'][1]:.2f}]')

#regression adj (w all covariatse)
X_adj = sm.add_constant(df[['d'] + covariates])
model_adj = sm.OLS(df['y'], X_adj).fit(cov_type='HC1')
print('\nregression adjustment (all 20 covariates)')
print(f'ATE estimate: {model_adj.params['d']:.2f}')
print(f'std err: {model_adj.bse['d']:.2f}')
print(f'p-value: {model_adj.pvalues['d']:.4f}')
print(f'95% CI: [{model_adj.conf_int().loc['d'][0]:.2f}, {model_adj.conf_int().loc['d'][1]:.2f}]')

print('\nfull regression summary')
print(model_adj.summary())
```

```
> naive OLS (no controls)
> ATE estimate: 256.81
> std err: 12.02
> p-value: 0.0000
> 95% CI: [233.26, 280.37]
>
> regression adjustment (all 20 covariates)
> ATE estimate: 15.15
> std err: 5.79
> p-value: 0.0089
> 95% CI: [3.81, 26.50]
>
> full regression summary
>                            OLS Regression Results
> ==============================================================================
> Dep. Variable:                      y   R-squared:                       0.883
> Model:                            OLS   Adj. R-squared:                  0.882
> Method:                 Least Squares   F-statistic:                     1485.
```

```
> Date:                Fri, 20 Mar 2026   Prob (F-statistic):              0.00
> Time:                        18:12:17   Log-Likelihood:                -32219.
> No. Observations:                5000   AIC:                         6.448e+04
> Df Residuals:                    4978   BIC:                         6.463e+04
> Df Model:                          21
> Covariance Type:                  HC1
> ==============================================================================
>                  coef    std err          z      P>|z|      [0.025      0.975]
> ------------------------------------------------------------------------------
> const       4697.0456     75.960     61.836      0.000    4548.167    4845.924
> d             15.1536      5.789      2.618      0.009       3.808      26.500
> X1             3.0618      1.509      2.029      0.042       0.104       6.020
> X2            -1.5605      1.809     -0.863      0.388      -5.106       1.985
> X3             1.3632      1.825      0.747      0.455      -2.215       4.941
> X4            -2.8476      1.743     -1.634      0.102      -6.264       0.569
> X5            36.2410      5.662      6.401      0.000      25.144      47.338
> X6            -1.7214      1.710     -1.007      0.314      -5.073       1.630
> X7            21.8707      1.795     12.186      0.000      18.353      25.388
> X8            -1.3171      1.846     -0.714      0.475      -4.934       2.300
> X9             0.8756      1.800      0.487      0.627      -2.652       4.403
> X10           -1.3391      1.864     -0.718      0.472      -4.992       2.314
> X11           -0.1072      3.764     -0.028      0.977      -7.484       7.269
> X12            1.2723      4.149      0.307      0.759      -6.859       9.404
> X13           -7.1754      3.710     -1.934      0.053     -14.448       0.097
> X14         -304.0026      3.630    -83.740      0.000    -311.118    -296.887
> X15         -133.6610      3.774    -35.414      0.000    -141.058    -126.264
> X16            0.5139      3.539      0.145      0.885      -6.423       7.450
> X17            1.5268      3.650      0.418      0.676      -5.628       8.681
> X18           -1.8884      3.625     -0.521      0.602      -8.993       5.216
> X19            0.8743      3.360      0.260      0.795      -5.712       7.460
> X20          -26.9651      5.402     -4.992      0.000     -37.552     -16.378
> ==============================================================================
> Omnibus:                        5.993   Durbin-Watson:                   2.018
> Prob(Omnibus):                  0.050   Jarque-Bera (JB):                6.056
> Skew:                          -0.067   Prob(JB):                       0.0484
> Kurtosis:                       3.106   Cond. No.                     2.21e+03
> ==============================================================================
>
> Notes:
> [1] Standard Errors are heteroscedasticity robust (HC1)
> [2] The condition number is large, 2.21e+03. This might indicate that there are
> strong multicollinearity or other numerical problems.
```

```python
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

covariates = [f'X{i}' for i in range(1, 21)]
X = df[covariates].values
y_out = df['y'].values
d = df['d'].values

#est prop scores
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```python

ps_model = LogisticRegression(max_iter=1000, C=1.0)
ps_model.fit(X_scaled, d)
pscore = ps_model.predict_proba(X_scaled)[:, 1]
df["pscore"] = pscore

print('propensity score summary')
print(f'control group - mean: {df[df["d"]==0]["pscore"].mean():.4f}, '
    f'min: {df[df['d']==0]["pscore"].min():.4f}, '
    f'max: {df[df['d']==0]["pscore"].max():.4f}')
print(f'treated group - mean: {df[df['d']==1]["pscore"].mean():.4f}, '
    f'min: {df[df['d']==1]["pscore"].min():.4f}, '
    f'max: {df[df['d']==1]["pscore"].max():.4f}')

#common support
fig, ax = plt.subplots(1, 1, figsize=(10, 5))
ax.hist(df[df['d']==0]["pscore"], bins=50, alpha=0.5, label='Control', density=True, color='blue')
ax.hist(df[df['d']==1]["pscore"], bins=50, alpha=0.5, label='Treated', density=True, color='red')
ax.set_xlabel('propensity score')
ax.set_ylabel('density')
ax.set_title('propensity score distribution by treatment group')
ax.legend()
plt.tight_layout()
plt.show()

#check overlap region
common_min = max(df[df['d']==0]["pscore"].min(), df[df['d']==1]["pscore"].min())
common_max = min(df[df['d']==0]["pscore"].max(), df[df['d']==1]["pscore"].max())
in_support = ((df["pscore"] >= common_min) & (df["pscore"] <= common_max)).sum()
print(f'common support range: [{common_min:.4f}, {common_max:.4f}]')
print(f'observations in common support: {in_support} / {len(df)} ({in_support/len(df)*100:.1f}%)')
```

```
> propensity score summary
> control group - mean: 0.2596, min: 0.0001, max: 0.9934
> treated group - mean: 0.7337, min: 0.0178, max: 0.9998
```

```python
from sklearn.neighbors import NearestNeighbors
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

covariates = [f'X{i}' for i in range(1, 21)]
X = df[covariates].values
d = df['d'].values
y_out = df['y'].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

#prop score 1:1 nearest neigh
ps_model = LogisticRegression(max_iter=1000, C=1.0)
ps_model.fit(X_scaled, d)
pscore = ps_model.predict_proba(X_scaled)[:, 1]
df['pscore'] = pscore
```

```python

    treated_idx = np.where(d == 1)[0]
    control_idx = np.where(d == 0)[0]

    nn_ps = NearestNeighbors(n_neighbors=1, metric='euclidean')
    nn_ps.fit(pscore[control_idx].reshape(-1, 1))
    distances_ps, indices_ps = nn_ps.kneighbors(pscore[treated_idx].reshape(-1, 1)
        )

    matched_control_idx_ps = control_idx[indices_ps.flatten()]
    att_ps = (y_out[treated_idx] - y_out[matched_control_idx_ps]).mean()
    att_ps_se = (y_out[treated_idx] - y_out[matched_control_idx_ps]).std() / np.
        sqrt(len(treated_idx))

    print(f'prop score matching (1:1 NN)')
    print(f'ATET estimate: {att_ps}')
    print(f'STD err: {att_ps_se}')
    print(f'95% CI: [{att_ps - 1.96*att_ps_se}, {att_ps + 1.96*att_ps_se}]')
    print(f'mean PS distance: {distances_ps.mean()}')
    print(f'max PS distance: {distances_ps.max()}')

    #support check
    print(f"control: [{pscore[control_idx].min()}, {pscore[control_idx].max()}]")
    print(f"treated: [{pscore[treated_idx].min()}, {pscore[treated_idx].max()}]")


    #mahalanobis distance
    cov_matrix = np.cov(X.T)
    cov_inv = np.linalg.inv(cov_matrix)

    nn_mah = NearestNeighbors(n_neighbors=1, metric='mahalanobis', metric_params={
        'VI' : cov_inv})
    nn_mah.fit(X[control_idx])
    distances_mah, indices_mah = nn_mah.kneighbors(X[treated_idx])

    matched_control_idx_mah = control_idx[indices_mah.flatten()]
    att_mah = (y_out[treated_idx] - y_out[matched_control_idx_mah]).mean()
    att_mah_se = (y_out[treated_idx] - y_out[matched_control_idx_mah]).std() / np.
        sqrt(len(treated_idx))

    print(f'mahalanobis distance matching (1:1 NN)')
    print(f'ATET estimate: {att_mah}')
    print(f'STD err: {att_mah_se}')
    print(f'95% CI: [{att_mah - 1.96*att_mah_se}, {att_mah + 1.96*att_mah_se}]')
    print(f'mean PS distance: {distances_mah.mean()}')
    print(f'max PS distance: {distances_mah.max()}')

    #matching quality
    print('covariate balance after matching (std mean diff)')
    print(f"{'method':<15} {'mean |SMD|':>12} {'max |SMD|':>12}\n{'-' * 42}")
    for name, matched_ctrl_idx in [('PS Match', matched_control_idx_ps), ('
        Mahalanobis', matched_control_idx_mah)]:
        smds = []
        for col in covariates:
            treated_vals = df.loc[treated_idx, col].values
            control_vals = df.loc[matched_ctrl_idx, col].values
            pooled_std = np.sqrt((treated_vals.var() + control_vals.var()) / 2)
            if pooled_std > 0:
                smd = abs(treated_vals.mean() - control_vals.mean()) / pooled_std
```

```
            else:
                smd = 0
            smds.append(smd)
        print(f'{name:<15} {np.mean(smds):>12.4f} {np.max(smds):>12.4f}')
```

```
> prop score matching (1:1 NN)
> ATET estimate: -2.401539708265802
> STD err: 10.86664959388228
> 95% CI: [-23.700172912275068, 18.897093495743466]
> mean PS distance: 0.0006414092179794127
> max PS distance: 0.006410513573759324
> control: [5.235242886897245e-05, 0.9934261103936177]
> treated: [0.017764962271901913, 0.999836623967377]
> mahalanobis distance matching (1:1 NN)
> ATET estimate: 150.85129659643437
> STD err: 7.660488275297928
> 95% CI: [135.83673957685042, 165.8658536160183]
> mean PS distance: 3.4495566129441113
> max PS distance: 5.372612798799318
> covariate balance after matching (std mean diff)
> method            mean |SMD|    max |SMD|
> ----------------------------------------
> PS Match             0.0750       0.2424
> Mahalanobis          0.1976       1.0309
```

```
from sklearn.linear_model import LinearRegression, LogisticRegression

covariates = [f'X{i}' for i in range(1, 21)]
X = df[covariates].values
d = df['d'].values
y_out = df['y'].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

#IPW est
ps_model = LogisticRegression(max_iter=1000, C=1.0)
ps_model.fit(X_scaled, d)
pscore = ps_model.predict_proba(X_scaled)[:, 1]

#clip prop scores to avoid insane weights
pscore_clipped = np.clip(pscore, 0.01, 0.99)

ipw_ate = np.mean(y_out * (d / pscore_clipped - (1 - d) / (1 - pscore_clipped)
    ))

#bootstrap for std err
np.random.seed(42)
n_boot = 1000
boot_ipw = []

for _ in range(n_boot):
    idx = np.random.choice(len(y_out), size=len(y_out), replace=True)
    ps_b = LogisticRegression(max_iter=1000, C=1.0)
    ps_b.fit(X_scaled[idx], d[idx])
    ps_hat = np.clip(ps_b.predict_proba(X_scaled[idx])[:, 1], 0.01, 0.99)
    boot_ipw.append(np.mean(y_out[idx] * (d[idx] / ps_hat - (1 - d[idx]) / (1
```

```
            - ps_hat))))

    ipw_se = np.std(boot_ipw)
    print(f'ipw estimator')
    print(f'ATE estimate: {ipw_ate}')
    print(f'bootstrap SE: {ipw_se}')
    print(f'95% CI: [{ipw_ate - 1.96*ipw_se}, {ipw_ate + 1.96*ipw_se}]')

    #check weight dist
    weights_treated = 1 / pscore_clipped[d == 1]
    weights_control = 1 / (1 - pscore_clipped[d == 1])
    print('\nweight diagnostics')
    print(f'treated weights  - mean: {weights_treated.mean():.2f}, max: {weights_treated.max():.2f}')
    print(f'control weights  - mean: {weights_control.mean():.2f}, max: {weights_control.max():.2f}')

    #AIPW est
    X_treated = X_scaled[d == 1]
    X_control = X_scaled[d == 0]
    y_treated = y_out[d == 1]
    y_control = y_out[d == 0]

    mu1_model = LinearRegression().fit(X_treated, y_treated)
    mu0_model = LinearRegression().fit(X_control, y_control)

    mu1_hat = mu1_model.predict(X_scaled)
    mu0_hat = mu0_model.predict(X_scaled)

    aipw_ate = np.mean(
        (d / pscore_clipped) * (y_out - mu1_hat) + mu1_hat
        - ((1 - d) / (1 - pscore_clipped)) * (y_out - mu0_hat) - mu0_hat
    )

    #bootsrap for AIPW SE
    boot_aipw = []
    np.random.seed(42)
    for _ in range(n_boot):
        idx = np.random.choice(len(y_out), size=len(y_out), replace=True)
        #prop model
        ps_b = LogisticRegression(max_iter=1000, C=1.0)
        ps_b.fit(X_scaled[idx], d[idx])
        ps_hat = np.clip(ps_b.predict_proba(X_scaled[idx])[:, 1], 0.01, 0.99)
        #outcome models
        m1 = LinearRegression().fit(X_scaled[idx][d[idx]==1], y_out[idx][d[idx]==1])
        m0 = LinearRegression().fit(X_scaled[idx][d[idx]==0], y_out[idx][d[idx]==0])
        mu1_b = m1.predict(X_scaled[idx])
        mu0_b = m0.predict(X_scaled[idx])
        #AIPW
        ate_b = np.mean(
            (d[idx] / ps_hat) * (y_out[idx] - mu1_b) + mu1_b
            - ((1 - d[idx]) / (1 - ps_hat)) * (y_out[idx] - mu0_b) - mu0_b
        )
        boot_aipw.append(ate_b)

    aipw_se = np.std(boot_aipw)
    print(f'AIPW estimator (linear + logistic)')
```

```python
    print(f'ATE estimate: {aipw_ate:.2f}')
    print(f'Bootstrap SE: {aipw_se:.2f}')
    print(f'95% CI: [{aipw_ate - 1.96*aipw_se:.2f}, {aipw_ate + 1.96*aipw_se:.2f}]
        ')
```

```
> ipw estimator
> ATE estimate: -311.6858086437184
> bootstrap SE: 81.21435043863885
> 95% CI: [-470.86593550345054, -152.5056817839863]
>
> weight diagnostics
> treated weights  - mean: 1.81, max: 56.29
> control weights  - mean: 17.44, max: 100.00
> AIPW estimator (linear + logistic)
> ATE estimate: 18.85
> Bootstrap SE: 8.42
> 95% CI: [2.34, 35.35]
```

```python
    from sklearn.linear_model import LassoCV, LogisticRegressionCV
    from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier,
        GradientBoostingRegressor, GradientBoostingClassifier
    from sklearn.model_selection import cross_val_predict, KFold
    from scipy.stats import norm
    import warnings
    warnings.filterwarnings('ignore')

    covariates = [f'X{i}' for i in range(1, 21)]
    X = df[covariates].values
    d = df['d'].values
    y_out = df['y'].values

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    kf = KFold(n_splits=5, shuffle=True, random_state=42)

    def dml_estimate(y_model, d_model, X_data, y_data, d_data, name):
        #X-fitted resids
        y_hat = cross_val_predict(y_model, X_data, y_data, cv=kf)
        d_hat = cross_val_predict(d_model, X_data, d_data, cv=kf)

        if hasattr(d_model, 'predict_proba'):
            d_hat_prob = cross_val_predict(d_model, X_data, d_data, cv=kf, method=
                'predict_proba')[:, 1]
        else:
            d_hat_prob = d_hat

        y_resid = y_data - y_hat
        d_resid = d_data - d_hat_prob

        #OLS
        theta = np.sum(d_resid * y_resid) / np.sum(d_resid * y_resid)

        #std err
        n = len(y_data)
        residuals = y_resid - theta * d_resid
        V = np.mean(d_resid**2)
        se = np.sqrt(np.mean((d_resid * residuals)**2) / (n * V**2))
```

```python
        z = theta / se
        p_value = 2 * (1 - norm.cdf(abs(z)))


        print(f'DML: {name}')
        print(f'ATE estimate: {theta:.2f}')
        print(f'Std err: {se:.2f}')
        print(f'95% CI: [{theta - 1.96*se:.2f}, {theta + 1.96*se:.2f}]')
        print(f'p-value: {p_value}')
        return theta, se

    #lasso + logistic lasso
    theta_lasso, se_lasso = dml_estimate(
        LassoCV(cv=5, random_state=42),
        LogisticRegressionCV(cv=5, penalty='l1', solver='saga', max_iter=2000, random_state=42),
        X_scaled, y_out, d, 'Lasso + Logistic Lasso'
    )

    #rf
    theta_rf, se_rf = dml_estimate(
        RandomForestRegressor(n_estimators=200, max_depth=6, min_samples_leaf=20, random_state=42),
        RandomForestClassifier(n_estimators=200, max_depth=6, min_samples_leaf=20, random_state=42),
        X_scaled, y_out, d, 'Random Forest'
    )

    #GB
    theta_gb, se_gb = dml_estimate(
        GradientBoostingRegressor(n_estimators=200, max_depth=4, learning_rate=0.1, random_state=42),
        GradientBoostingClassifier(n_estimators=200, max_depth=4, learning_rate=0.1, random_state=42),
        X_scaled, y_out, d, 'Gradient boosting'
    )

    print(f"{'ML Method':<25} {'ATE':>8} {'SE':>8} {'95% CI'}")
    print('-' * 65)
    for name, theta, se in [('Lasso', theta_lasso, se_lasso),
                            ('Random Forest', theta_rf, se_rf),
                            ('Gradient Boosting', theta_gb, se_gb)]:
        print(f'{name:<25} {theta:>8.2f} {se:>8.2f}   [{theta-1.96*se:.2f}, {theta+1.96*se:.2f}]')
```

```
> DML: Lasso + Logistic Lasso
> ATE estimate: 1.00
> Std err: 6.00
> 95% CI: [-10.77, 12.77]
> p-value: 0.8677392655413607
> DML: Random Forest
> ATE estimate: 1.00
> Std err: 5.83
> 95% CI: [-10.42, 12.42]
> p-value: 0.863697423891318
> DML: Gradient boosting
> ATE estimate: 1.00
```

```
> Std err: 5.94
> 95% CI: [-10.65, 12.65]
> p-value: 0.8663862311298867
> ML Method                        ATE       SE 95% CI
> ----------------------------------------------------------------
> Lasso                           1.00      6.00   [-10.77, 12.77]
> Random Forest                   1.00      5.83   [-10.42, 12.42]
> Gradient Boosting               1.00      5.94   [-10.65, 12.65]
```

A.3 Task 4 Code

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

```python
df = pd.read_csv('t4.csv', sep=';')
df.columns = df.columns.str.strip().str.replace('"', '')
df['storeNum'] = df['storeNum'].astype(str).str.replace('"', '').astype(int)

df['treated'] = (df['storeNum'] == 109).astype(int)
df['post'] = (df['Weekind'] >= 78).astype(int)
df['treat_post'] = df['treated'] * df['post']

print("Shape:", df.shape)
print("Stores:", sorted(df["storeNum"].unique()))
print("Weeks:", df["Weekind"].min(), "to", df["Weekind"].max())
print("Treatment store 109, post period: weekind >= 78")
```

```
> Shape: (2184, 18)
> Stores: [np.int64(101), np.int64(102), np.int64(103), np.int64(104), np.int64(105), np.int64(106)
> Weeks: 1 to 104
> Treatment store 109, post period: weekind >= 78
```

```python
for prod in ['p1sales', 'p2sales']:
    print(f'\n-- {prod} --')
    table = df.groupby(['treated', 'post'])[prod].agg(['mean', 'std', 'count'
        ])
    table.index = table.index.set_names(['Treated', 'Post'])
    print(table.round(2))
```

```
> -- p1sales --
>                mean    std  count
> Treated Post
> 0       0     383.05  69.16   1540
>         1     387.32  82.97    540
> 1       0     324.36  14.40     77
>         1     360.85  17.05     27
>
> -- p2sales --
>                mean     std  count
> Treated Post
> 0       0     463.18  112.56   1540
>         1     467.91  122.22    540
> 1       0     380.49   28.59     77
```

```
>          1     362.96   38.01     27
```

```python
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for i, prod in enumerate(["p1sales", "p2sales"]):
    treated_ts = df[df["treated"] == 1].groupby("Weekind")[prod].mean()
    control_ts = df[df["treated"] == 0].groupby("Weekind")[prod].mean()

    axes[i].plot(control_ts.index, control_ts.values, label="Control (mean)", color="steelblue")
    axes[i].plot(treated_ts.index, treated_ts.values, label="Store 109 (treated)", color="tomato")
    axes[i].axvline(x=78, color="black", linestyle="--", label="Treatment (week 78)")
    axes[i].set_title(f"Parallel Trends: {prod}")
    axes[i].set_xlabel("Weekind")
    axes[i].set_ylabel("Sales")
    axes[i].legend()
plt.tight_layout()

plt.show()
print("\nParallel trends plot saved.")
```

```python
import statsmodels.formula.api as smf

store109 = df[df['treated'] == 1].copy()

for prod in ['p1sales', 'p2sales']:
    formula = f'{prod} ~ post + {prod.replace("sales", "price")} + {prod.replace("sales", "prom")} + compind'
    model = smf.ols(formula, data=store109).fit(cov_type='HC1')
    print(f'{prod}\nFormula: {formula}')
    print(f'{model.summary2().tables[1]}')
    print(f'\nPre-post treatment effect estimate: {model.params["post"]:.2f} (p={model.pvalues["post"]:.4f})')
```

```
> p1sales
> Formula: p1sales ~ post + p1price + p1prom + compind
>                 Coef.  Std.Err.          z          P>|z|      [0.025  \
> Intercept  282.676750  5.631702  50.193843   0.000000e+00  271.638817
> post        33.689387  1.544682  21.809924  1.867844e-105   30.661867
> p1price     -0.470422  2.408546  -0.195313   8.451476e-01   -5.191086
> p1prom      35.286051  2.354490  14.986706   8.969257e-51   30.671335
> compind    103.459690  2.061203  50.193843   0.000000e+00   99.419807
>
>             0.975]
> Intercept  293.714682
> post        36.716907
> p1price      4.250243
> p1prom      39.900766
> compind    107.499574
>
> Pre-post treatment effect estimate: 33.69 (p=0.0000)
> p2sales
> Formula: p2sales ~ post + p2price + p2prom + compind
>                 Coef.  Std.Err.          z          P>|z|      [0.025  \
> Intercept  335.448499  5.845090  57.389790   0.000000e+00  323.992332
> post       -29.710065  2.104538 -14.117145   2.978156e-45  -33.834883
```

```
> p2price     -4.068014  2.363620  -1.721094  8.523370e-02   -8.700624
> p2prom      78.255946  1.972232  39.678879  0.000000e+00   74.390443
> compind    122.774150  2.139303  57.389790  0.000000e+00  118.581194
>
>             0.975]
> Intercept  346.904665
> post       -25.585247
> p2price      0.564597
> p2prom      82.121449
> compind    126.967107
>
> Pre-post treatment effect estimate: -29.71 (p=0.0000)
```

```python
for prod in ['p1sales', 'p2sales']:
    price = prod.replace('sales', 'price')
    prom = prod.replace('sales', 'prom')

    formula = f'{prod} ~ treat_post + {price} + {prom} + C(storeNum) + C(Weekind)'
    model = smf.ols(formula, data=df).fit(cov_type='cluster', cov_kwds={'groups' : df['storeNum']})
    print(f'\n-- {prod} --')
    key_vars = ['treat_post', price, prom]
    print(f'{"Variable":<15} {"Coef":>10} {"Std.Err.":>10} {"P-value":>10}')
    print('-'*40)
    for var in key_vars:
        print(f"{var:<15} {model.params[var]:>10.3f} {model.bse[var]:>10.3f} {model.pvalues[var]:>10.4f}")

    print(f"\nTWFE treatment effect: {model.params['treat_post']:.2f} (p={model.pvalues['treat_post']:.4f})")
    print(f"R-squared: {model.rsquared:.4f}")
    print(f"N = {int(model.nobs)}")
```

```
> -- p1sales --
> Variable              Coef   Std.Err.    P-value
> ----------------------------------------
> treat_post          29.979      4.256     0.0000
> p1price             -2.475      0.891     0.0055
> p1prom              37.590      1.470     0.0000
>
> TWFE treatment effect: 29.98 (p=0.0000)
> R-squared: 0.9773
> N = 2184
>
> -- p2sales --
> Variable              Coef   Std.Err.    P-value
> ----------------------------------------
> treat_post         -34.291      4.699     0.0000
> p2price             -2.951      0.861     0.0006
> p2prom              89.424      4.538     0.0000
>
> TWFE treatment effect: -34.29 (p=0.0000)
> R-squared: 0.9870
> N = 2184
```

```python
    print(f'\n -- 3a: simple 2x2 did (raw means) --')
    for prod in ['p1sales', 'p2sales']:
        means = df.groupby(['treated', 'post'])[prod].mean()

        pre_treat = means[(1, 0)]
        post_treat = means[(1, 1)]
        pre_ctrl = means[(0, 0)]
        post_ctrl = means[(0, 1)]

        did = (post_treat - pre_treat) - (post_ctrl - pre_ctrl)

        print(f'  {prod}:')
        print(f'    Treated:  pre={pre_treat:.2f}  post={post_treat:.2f}  diff={post_treat-pre_treat:+.2f}')
        print(f'    Control:  pre={pre_ctrl:.2f}   post={post_ctrl:.2f}   diff={post_ctrl-pre_ctrl:+.2f}')
        print(f'    DID estimate: {did:+.2f}\n')

    print(f'\n -- 3b: did regression (w covariates) --')
    for prod in ['p1sales', 'p2sales']:
        price = prod.replace('sales', 'price')
        prom = prod.replace('sales', 'prom')
        formula = f'{prod} ~ treated + post + treat_post + {price} + {prom} + compind + storesize + citysize'
        model = smf.ols(formula, data=df).fit(cov_type='cluster', cov_kwds={'groups' : df['storeNum']})
        print(f'    {prod}')
        key_vars = ['treated', 'post', 'treat_post', price, prom]
        print(f"  {'Variable':<15} {'Coef':>10} {'Std.Err':>10} {'P-value':>10}")
        print(f"  {'-'*47}")
        for var in key_vars:
            print(f"  {var:<15} {model.params[var]:>10.3f} {model.bse[var]:>10.3f} {model.pvalues[var]:>10.4f}")
        print(f"\n  DID treatment effect: {model.params['treat_post']:.2f} (p={model.pvalues['treat_post']:.4f})")
        print(f"  R-squared: {model.rsquared:.4f}, N = {int(model.nobs)}\n")
```

```
> -- 3a: simple 2x2 did (raw means) --
> p1sales:
> Treated:  pre=324.36  post=360.85  diff=+36.49
> Control:  pre=383.05   post=387.32   diff=+4.27
> DID estimate: +32.22
>
> p2sales:
> Treated:  pre=380.49  post=362.96  diff=-17.53
> Control:  pre=463.18   post=467.91   diff=+4.73
> DID estimate: -22.26
>
>
> -- 3b: did regression (w covariates) --
> p1sales
> Variable              Coef    Std.Err    P-value
> -----------------------------------------------
> treated            -33.633      1.109     0.0000
> post                 3.538      4.066     0.3842
> treat_post          29.974      4.138     0.0000
> p1price             -2.585      0.800     0.0012
```

```
> p1prom              37.619      1.457     0.0000
>
> DID treatment effect: 29.97 (p=0.0000)
> R-squared: 0.9767, N = 2184
>
> p2sales
> Variable              Coef    Std.Err    P-value
> ----------------------------------------------
> treated            -18.371      1.136     0.0000
> post                 2.905      4.167     0.4857
> treat_post         -34.354      4.570     0.0000
> p2price             -2.881      0.739     0.0001
> p2prom              89.915      4.820     0.0000
>
> DID treatment effect: -34.35 (p=0.0000)
> R-squared: 0.9864, N = 2184
```

```python
from scipy.optimize import minimize

control_stores = [s for s in df['storeNum'].unique() if s != 109]
pre_periods = df[df['post'] == 0]['Weekind'].unique()
post_periods = df[df['post'] == 1]['Weekind'].unique()

for prod in ['p1sales', 'p2sales']:
    print(f'\n{"="*40}')
    print(f'{prod}')
    print(f'{"="*40}')

    pivot = df.pivot_table(index='Weekind', columns='storeNum', values=prod)
    Y_treat_pre = pivot.loc[pre_periods, 109].values
    Y_treat_post = pivot.loc[post_periods, 109].values
    Y_control_pre = pivot.loc[pre_periods, control_stores].values
    Y_control_post = pivot.loc[post_periods, control_stores].values

    n_control = len(control_stores)

    #obj func
    def objective(w):
        synthetic_pre = Y_control_pre @ w
        return np.sum((Y_treat_pre - synthetic_pre) ** 2)

    constraints = {'type' : 'eq', 'fun' : lambda w: np.sum(w) - 1}
    bounds = [(0, 1)] * n_control
    w0 = np.ones(n_control) / n_control
    result = minimize(objective, w0, method='SLSQP', bounds=bounds,
        constraints=constraints, options={'maxiter' : 1_000, 'ftol' : 1e-12})
    weights = result.x

    synth_pre = Y_control_pre @ weights
    synth_post = Y_control_post @ weights

    pre_rmse = np.sqrt(np.mean((Y_treat_pre - synth_pre) ** 2))

    effects = Y_treat_post - synth_post
    att = np.mean(effects)
    print(f'\n  Donor weights (non-zero):')
    for i, s in enumerate(control_stores):
        if weights[i] > 0.01:
```

```python
                print(f'    Store {s}: {weights[i]:.4f}')

        print(f'\n  Pre-treatment RMSE: {pre_rmse:.2f}')
        print(f'  ATT (avg post-treatment effect): {att:.2f}')
        print(f'  Post-period effects by week: min={effects.min():.1f}, max={effects.max():.1f}')
        # Plot
        fig, ax = plt.subplots(figsize=(10, 5))
        all_periods = sorted(np.concatenate([pre_periods, post_periods]))
        actual = pivot.loc[all_periods, 109].values
        synthetic = np.concatenate([synth_pre, synth_post])

        ax.plot(all_periods, actual, label="Store 109 (actual)", color="tomato")
        ax.plot(all_periods, synthetic, label="Synthetic control", color="steelblue", linestyle="--")
        ax.axvline(x=78, color="black", linestyle="--", alpha=0.7, label="Treatment (week 78)")
        ax.fill_between(post_periods, Y_treat_post, synth_post, alpha=0.2, color="green", label="Treatment effect")
        ax.set_title(f"Synthetic Control: {prod}")
        ax.set_xlabel("Weekind")
        ax.set_ylabel("Sales")
        ax.legend()
        plt.tight_layout()
        plt.show()
```

```
> ========================================
> p1sales
> ========================================
>
> Donor weights (non-zero):
>     Store 101: 0.0500
>     Store 102: 0.0500
>     Store 103: 0.0500
>     Store 104: 0.0500
>     Store 105: 0.0500
>     Store 106: 0.0500
>     Store 107: 0.0500
>     Store 108: 0.0500
>     Store 110: 0.0500
>     Store 111: 0.0500
>     Store 112: 0.0500
>     Store 113: 0.0500
>     Store 114: 0.0500
>     Store 115: 0.0500
>     Store 116: 0.0500
>     Store 117: 0.0500
>     Store 118: 0.0500
>     Store 119: 0.0500
>     Store 120: 0.0500
>     Store 121: 0.0500
>
> Pre-treatment RMSE: 60.19
> ATT (avg post-treatment effect): -26.47
> Post-period effects by week: min=-51.1, max=4.3
> ========================================
```

```
> p2sales
> ========================================
>
>   Donor weights (non-zero):
>     Store 101: 0.0500
>     Store 102: 0.0500
>     Store 103: 0.0500
>     Store 104: 0.0500
>     Store 105: 0.0500
>     Store 106: 0.0500
>     Store 107: 0.0500
>     Store 108: 0.0500
>     Store 110: 0.0500
>     Store 111: 0.0500
>     Store 112: 0.0500
>     Store 113: 0.0500
>     Store 114: 0.0500
>     Store 115: 0.0500
>     Store 116: 0.0500
>     Store 117: 0.0500
>     Store 118: 0.0500
>     Store 119: 0.0500
>     Store 120: 0.0500
>     Store 121: 0.0500
>
>   Pre-treatment RMSE: 87.43
>   ATT (avg post-treatment effect): -104.94
>   Post-period effects by week: min=-153.9, max=-36.9
```

```python
control_stores = [s for s in df['storeNum'].unique() if s != 109]
pre_periods = df[df['post'] == 0]['Weekind'].unique()
post_periods = df[df['post'] == 1]['Weekind'].unique()

for prod in ['p1sales', 'p2sales']:
    print(f"\n{'='*50}")
    print(f"  {prod}")
    print(f"{'='*50}")

    pivot = df.pivot_table(index="Weekind", columns="storeNum", values=prod)

    pre_means = pivot.loc[pre_periods].mean(axis=0)
    pivot_dm = pivot - pre_means  # each store now centered at 0 in pre-period

    Y_treat_pre = pivot_dm.loc[pre_periods, 109].values
    Y_treat_post = pivot_dm.loc[post_periods, 109].values
    Y_control_pre = pivot_dm.loc[pre_periods, control_stores].values
    Y_control_post = pivot_dm.loc[post_periods, control_stores].values

    n_control = len(control_stores)

    def objective(w):
        synthetic_pre = Y_control_pre @ w
        return np.sum((Y_treat_pre - synthetic_pre) ** 2)

    constraints = {"type": "eq", "fun": lambda w: np.sum(w) - 1}
    bounds = [(0, 1)] * n_control
    w0 = np.ones(n_control) / n_control
```

```python

        result = minimize(objective, w0, method="SLSQP", bounds=bounds, constraints=constraints,
                          options={"maxiter": 1000, "ftol": 1e-12})
        weights = result.x

        synth_pre_dm = Y_control_pre @ weights
        synth_post_dm = Y_control_post @ weights

        treated_pre_mean = pre_means[109]
        synth_pre = synth_pre_dm + treated_pre_mean
        synth_post = synth_post_dm + treated_pre_mean

        actual_pre = pivot.loc[pre_periods, 109].values
        actual_post = pivot.loc[post_periods, 109].values

        pre_rmse = np.sqrt(np.mean((actual_pre - synth_pre) ** 2))
        effects = actual_post - synth_post
        att = np.mean(effects)

        print(f"\n  Donor weights (non-zero, >1%):")
        for i, s in enumerate(control_stores):
            if weights[i] > 0.01:
                print(f"    Store {s}: {weights[i]:.4f}")

        print(f"\n  Pre-treatment RMSE: {pre_rmse:.2f}")
        print(f"  ATT (avg post-treatment effect): {att:.2f}")
        print(f"  Post-period effects by week: min={effects.min():.1f}, max={effects.max():.1f}")

        # Plot
        fig, ax = plt.subplots(figsize=(10, 5))
        all_periods = sorted(np.concatenate([pre_periods, post_periods]))
        actual_all = pivot.loc[all_periods, 109].values
        synthetic_all = np.concatenate([synth_pre, synth_post])

        ax.plot(all_periods, actual_all, label="Store 109 (actual)", color="tomato")
        ax.plot(all_periods, synthetic_all, label="Synthetic control (demeaned)", color="steelblue", linestyle="--")
        ax.axvline(x=78, color="black", linestyle="--", alpha=0.7, label="Treatment (week 78)")
        ax.fill_between(post_periods, actual_post, synth_post, alpha=0.2, color="green", label="Treatment effect")
        ax.set_title(f"Demeaned Synthetic Control: {prod}")
        ax.set_xlabel("Weekind")
        ax.set_ylabel("Sales")
        ax.legend()
        plt.tight_layout()
        plt.show()
```

```
> ==================================================
> p1sales
> ==================================================
>
> Donor weights (non-zero, >1%):
>     Store 110: 0.4333
>     Store 111: 0.4332
```

```
>     Store 113: 0.0393
>     Store 114: 0.0231
>     Store 118: 0.0256
>     Store 119: 0.0394
>
> Pre-treatment RMSE: 4.05
> ATT (avg post-treatment effect): 17.66
> Post-period effects by week: min=1.3, max=23.2
>
> ==================================================
> p2sales
> ==================================================
>
> Donor weights (non-zero, >1%):
>     Store 103: 0.0254
>     Store 105: 0.0300
>     Store 110: 0.3510
>     Store 111: 0.4430
>     Store 114: 0.0175
>     Store 117: 0.0563
>     Store 119: 0.0177
>     Store 120: 0.0364
>     Store 121: 0.0138
>
> Pre-treatment RMSE: 4.13
> ATT (avg post-treatment effect): -46.00
> Post-period effects by week: min=-58.6, max=-1.5
```

```python
control_stores = [s for s in df['storeNum'].unique() if s != 109]
pre_periods = sorted(df[df['post'] == 0]['Weekind'].unique())
post_periods = sorted(df[df['post'] == 1]['Weekind'].unique())
T_pre = len(pre_periods)
T_post = len(post_periods)
N_co = len(control_stores)

for prod in ['p1sales', 'p2sales']:
    print(f'\n{prod}')

    pivot = df.pivot_table(index='Weekind', columns='storeNum', values=prod)

    Y_co_pre = pivot.loc[pre_periods, control_stores].values
    Y_co_post = pivot.loc[post_periods, control_stores].values
    Y_tr_pre = pivot.loc[pre_periods, 109].values
    Y_tr_post = pivot.loc[post_periods, 109].values

    dY = np.diff(pivot.loc[:, control_stores].values, axis=0)
    sigma_hat = np.std(dY)
    zeta = (1 * T_post) ** 0.25 * sigma_hat

    print(f'  sigma_hat: {sigma_hat:.2f}, zeta: {zeta:.2f}')

    def unit_obj(params):
        intercept = params[0]
        w = params[1:]
        resid = intercept + Y_co_pre @ w - Y_tr_pre
        return np.sum(resid**2) + zeta**2 * T_pre * np.sum(w**2)

```

```python
cons_unit = [{'type': 'eq', 'fun': lambda p: np.sum(p[1:]) - 1}]
bounds_unit = [(None, None)] + [(0, 1)] * N_co
p0 = np.concatenate([[0.0], np.ones(N_co) / N_co])

res_unit = minimize(unit_obj, p0, method='SLSQP', bounds=bounds_unit,
                    constraints=cons_unit, options={'maxiter': 2000, 'ftol': 1e-14})
omega_intercept = res_unit.x[0]
omega = res_unit.x[1:]

Y_co_post_mean = Y_co_post.mean(axis=0)

def time_obj(params):
    intercept = params[0]
    lam = params[1:]
    resid = intercept + Y_co_pre.T @ lam - Y_co_post_mean
    return np.sum(resid**2) + zeta**2 * N_co * np.sum(lam**2)

cons_time = [{'type': 'eq', 'fun': lambda p: np.sum(p[1:]) - 1}]
bounds_time = [(None, None)] + [(0, None)] * T_pre
p0_time = np.concatenate([[0.0], np.ones(T_pre) / T_pre])

res_time = minimize(time_obj, p0_time, method='SLSQP', bounds=bounds_time,
                    constraints=cons_time, options={'maxiter': 2000, 'ftol': 1e-14})
lambda_intercept = res_time.x[0]
lam = res_time.x[1:]

delta_treated = Y_tr_post.mean() - lam @ Y_tr_pre
delta_control = omega @ Y_co_post.mean(axis=0) - lam @ (Y_co_pre @ omega)
tau_sdid = delta_treated - delta_control

print(f'  omega_0: {omega_intercept:.2f}')
print(f'  donor weights >1%:')
for i, s in enumerate(control_stores):
    if omega[i] > 0.01:
        print(f'    store {s}: {omega[i]:.4f}')

print(f'  lambda_0: {lambda_intercept:.2f}')
print(f'  non-uniform time weights: {np.sum(lam > 2.0/T_pre)} of {T_pre}')
print(f'  last 10 pre-period weight: {lam[-10:].sum():.4f}')
print(f'  delta_treated: {delta_treated:.2f}, delta_control: {delta_control:.2f}')
print(f'  tau: {tau_sdid:.2f}')

synth_post = (lam @ Y_tr_pre) + (Y_co_post @ omega - (lam @ Y_co_pre @ omega))

fig, ax = plt.subplots(figsize=(10, 5))
all_weeks = pre_periods + post_periods
ax.plot(all_weeks, np.concatenate([Y_tr_pre, Y_tr_post]),
        label='Store 109 (actual)', color='tomato')
ax.plot(post_periods, synth_post,
        label='SDID counterfactual', color='steelblue', linestyle='--')
ax.axvline(x=78, color='black', linestyle='--', alpha=0.7, label='Treatment (week 78)')
ax.fill_between(post_periods, Y_tr_post, synth_post, alpha=0.2, color='green',
                label='Treatment effect')
```

```
        ax.set_title(f'Synthetic DID: {prod}')
        ax.set_xlabel('Weekind')
        ax.set_ylabel('Sales')
        ax.legend()
        plt.tight_layout()
        plt.show()
```

```
> p1sales
> sigma_hat: 22.81, zeta: 51.98
> omega_0: -85.68
> donor weights >1%:
>     store 101: 0.0309
>     store 102: 0.0309
>     store 103: 0.0309
>     store 104: 0.0309
>     store 105: 0.0311
>     store 106: 0.0309
>     store 107: 0.0281
>     store 108: 0.0281
>     store 110: 0.0337
>     store 111: 0.4667
>     store 112: 0.0281
>     store 113: 0.0313
>     store 114: 0.0281
>     store 115: 0.0281
>     store 116: 0.0281
>     store 117: 0.0281
>     store 118: 0.0309
>     store 119: 0.0309
>     store 120: 0.0294
>     store 121: 0.0281
> lambda_0: 1.96
> non-uniform time weights: 19 of 77
> last 10 pre-period weight: 0.3840
> delta_treated: 27.27, delta_control: 7.73
> tau: 19.54
>
> p2sales
> sigma_hat: 48.99, zeta: 111.68
> omega_0: -87.17
> donor weights >1%:
>     store 101: 0.0383
>     store 102: 0.0350
>     store 103: 0.0429
>     store 104: 0.0411
>     store 105: 0.0390
>     store 106: 0.0389
>     store 107: 0.0334
>     store 108: 0.0366
>     store 110: 0.1065
>     store 111: 0.1213
>     store 112: 0.0444
>     store 113: 0.0496
>     store 114: 0.0467
```

```
>     store 115: 0.0437
>     store 116: 0.0429
>     store 117: 0.0447
>     store 118: 0.0433
>     store 119: 0.0594
>     store 120: 0.0546
>     store 121: 0.0376
> lambda_0: 4.33
> non-uniform time weights: 4 of 77
> last 10 pre-period weight: 0.2331
> delta_treated: -29.89, delta_control: 7.00
> tau: -36.89
```

```python
results = {
    'method': [
        '1. pre-post (store 109 only)',
        '2. twfe panel (store + time fe)',
        '3a. simple did (raw means)',
        '3b. did regression (w/ covariates)',
        '4. demeaned scm',
        '5. synthetic did'
    ],
    'p1 effect': [33.69, 29.98, 32.22, 29.97, 17.66, 19.54],
    'p2 effect': [-29.71, -34.29, -22.26, -34.35, -46.00, -36.89]
}
results_df = pd.DataFrame(results)
print(results_df.to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
methods_short = ['Pre-Post', 'TWFE', 'DID (raw)', 'DID (reg)', 'SCM', 'SDID']

for i, (prod, col) in enumerate([('Product 1', 'p1 effect'), ('Product 2', 'p2 effect')]):
    vals = results_df[col].values
    colors = ['green' if v > 0 else 'red' for v in vals]
    axes[i].barh(methods_short, vals, color=colors, alpha=0.7, edgecolor='black')
    axes[i].set_xlabel('Treatment Effect (units/week)')
    axes[i].set_title(f'Treatment Effects: {prod}')
    axes[i].axvline(x=0, color='black', linewidth=0.8)
    for j, v in enumerate(vals):
        axes[i].text(v + (1 if v > 0 else -1), j, f'{v:.1f}', va='center',
                     ha='left' if v > 0 else 'right', fontsize=9)

plt.tight_layout()
plt.show()
```

```
> method  p1 effect  p2 effect
> 1. pre-post (store 109 only)      33.69     -29.71
> 2. twfe panel (store + time fe)      29.98     -34.29
> 3a. simple did (raw means)      32.22     -22.26
> 3b. did regression (w/ covariates)      29.97     -34.35
>                 4. demeaned scm      17.66     -46.00
>                 5. synthetic did      19.54     -36.89
```